Politechnika Świętokrzyska
Wydział Elektrotechniki, Automatyki i Informatyki

Niezawodność systemów komputerowych

Sprawozdanie z zadania laboratoryjnego

Temat: Wyznaczenie empirycznych wskaźników niezawodności $F^*(t)$, $R^*(t)$, $f^*(t)$, $\lambda^*(t)$, E^*T na podstawie próby losowej oraz ręcznie wprowadzonych danych

Autorzy: Filip Stępień
Bartłomiej Karkoszka
Rafał Grot

Grupa: 2ID21A

Rok akademicki: 2026/2027

Spis treści

1 Opis teoretyczny zagadnienia **3**
- 1.1 Trwałość elementu jako zmienna losowa . . . 3
- 1.2 Dane z badania . . . 3
- 1.3 Dystrybuanta . . . 3
- 1.4 Funkcja niezawodności . . . 3
- 1.5 Gęstość prawdopodobieństwa . . . 4
- 1.6 Intensywność uszkodzeń . . . 4
- 1.7 Wartość oczekiwana trwałości . . . 5
- 1.8 Próba losowa . . . 5
- 1.9 Grupowanie pojedynczych czasów uszkodzeń . . . 6

2 Algorytm działania programu **7**
- 2.1 Ogólny przebieg . . . 7
- 2.2 Wyznaczanie wskaźników . . . 7
- 2.3 Grupowanie czasów uszkodzeń . . . 8
- 2.4 Losowanie próby . . . 8

3 Kod programu **10**
- 3.1 Uruchomienie programu . . . 10
- 3.2 Wyznaczanie wskaźników . . . 10
- 3.3 Rozkłady trwałości . . . 15
- 3.4 Obsługa trzech sposobów wprowadzania danych . . . 18
- 3.5 Wczytywanie danych . . . 20
- 3.6 Zapis liczb . . . 21
- 3.7 Wykresy . . . 22
- 3.8 Zapis plików . . . 27
- 3.9 Okno programu . . . 28

4 Opis i wyniki testowania aplikacji **45**
- 4.1 Testy automatyczne . . . 45
- 4.2 Zgodność z przykładami z książki . . . 45
- 4.3 Próba losowa a rozkład teoretyczny . . . 47
- 4.4 Wpływ długości przedziału na wyniki . . . 49
- 4.5 Próba niepełna . . . 50
- 4.6 Obsługa błędnych danych . . . 51
- 4.7 Wnioski . . . 52

5 Instrukcja obsługi programu **53**
- 5.1 Uruchomienie . . . 53
- 5.2 Okno programu . . . 53
- 5.3 Próba losowa . . . 53
- 5.4 Tabela uszkodzeń . . . 54
- 5.5 Lista czasów . . . 54
- 5.6 Wyniki . . . 54
- 5.7 Wykresy . . . 55
- 5.8 Próba . . . 56
- 5.9 Zapis wyników . . . 57

5.10 Komunikaty o błędach 57
5.11 Budowanie programu z kodu źródłowego 57

1 Opis teoretyczny zagadnienia

Opis oparto na rozdziale 5 książki [1] (s. 208–213); numery wzorów w nawiasach odpowiadają numeracji z tej książki.

1.1 Trwałość elementu jako zmienna losowa

Trwałość T elementu, czyli czas jego poprawnej pracy do chwili uszkodzenia, jest nieujemną zmienną losową. Opisują ją cztery funkcje ciągłe względem czasu, określone dla $t \geq 0$: dystrybuanta $F(t)$, funkcja niezawodności $R(t)$, gęstość prawdopodobieństwa $f(t)$ oraz funkcja intensywności uszkodzeń $\lambda(t)$. Każda z nich określa zmienną T jednoznacznie, a więc wyznacza także pozostałe trzy. Najważniejsze związki między nimi to:

$$R(t) = 1 - F(t), \qquad f(t) = \frac{\mathrm{d}F(t)}{\mathrm{d}t} = -\frac{\mathrm{d}R(t)}{\mathrm{d}t},$$
$$\lambda(t) = \frac{f(t)}{R(t)}, \qquad R(t) = \exp\Big(-\int_0^t \lambda(\tau)\,\mathrm{d}\tau\Big).$$

W praktyce funkcje te nie są znane i szacuje się je na podstawie badania próbki losowej. Wskaźniki wyznaczone z próbki nazywa się empirycznymi i oznacza gwiazdką.

1.2 Dane z badania

W chwili $t = 0$ badaniu poddaje się $n(0)$ zdatnych elementów. W kolejnych chwilach $t_1 < t_2 < \cdots < t_k$ rejestruje się liczbę $m(t_j)$ elementów, które uległy uszkodzeniu od poprzedniej rejestracji, czyli w przedziale $(t_{j-1}, t_j]$, przy czym $t_0 = 0$. Długość przedziału oznacza się $\Delta t_j = t_j - t_{j-1}$. Liczba elementów zdatnych w chwili t_i wynosi

$$n(t_i) = n(0) - \sum_{j=1}^{i} m(t_j).$$

1.3 Dystrybuanta

Dystrybuanta trwałości jest prawdopodobieństwem uszkodzenia elementu do chwili t:

$$F(t) = \Pr(T < t), \qquad t \geq 0, \tag{5-1}$$

przy czym zwykle przyjmuje się $F(0) = 0$. Dystrybuantę empiryczną wyznacza się jako udział elementów uszkodzonych do chwili t_i w całej próbce:

$$F^*(t_i) = \frac{1}{n(0)} \sum_{j=1}^{i} m(t_j). \tag{5-2}$$

1.4 Funkcja niezawodności

Funkcja niezawodności jest prawdopodobieństwem, że element zdatny w chwili $t = 0$ pozostanie zdatny w całym przedziale $(0, t)$:

$$R(t) = \Pr(T > t), \qquad t \geq 0, \tag{5-3}$$

$$R(t) = 1 - F(t), \qquad R(0) = 1. \tag{5-4}$$

Empiryczna funkcja niezawodności to udział elementów nadal zdatnych w chwili t_i:

$$R^*(t_i) = \frac{n(t_i)}{n(0)} = \frac{n(0) - \sum_{j=1}^{i} m(t_j)}{n(0)}. \tag{5-5}$$

1.5 Gęstość prawdopodobieństwa

Gęstość prawdopodobieństwa trwałości jest zdefiniowana jako

$$f(t) = \lim_{\Delta t \to 0} \frac{\Pr(t < T < t + \Delta t)}{\Delta t}, \tag{5-6}$$

skąd

$$F(t) = \int_0^t f(\tau)\,\mathrm{d}\tau, \qquad R(t) = \int_t^\infty f(\tau)\,\mathrm{d}\tau, \tag{5-7}$$

$$f(t) = \frac{\mathrm{d}F(t)}{\mathrm{d}t} = -\frac{\mathrm{d}R(t)}{\mathrm{d}t}. \tag{5-8}$$

Empiryczną gęstość wyznacza się, odnosząc liczbę uszkodzeń w przedziale do *początkowej* liczności próbki i do długości przedziału:

$$f^*(t) = \frac{\Delta m(t, \Delta t)}{n(0)\,\Delta t}, \tag{5-9}$$

gdzie $\Delta m(t, \Delta t)$ jest liczbą elementów uszkodzonych w przedziale $[t, t + \Delta t]$. Dla danych rejestrowanych w chwilach t_i oznacza to

$$f^*(t_i) = \frac{m(t_i)}{n(0)\,(t_i - t_{i-1})}.$$

1.6 Intensywność uszkodzeń

Funkcja intensywności uszkodzeń mówi o szybkości uszkadzania się elementów, które dotrwały do chwili t:

$$\lambda(t) = \lim_{\Delta t \to 0} \frac{\Pr(t < T \leq t + \Delta t \mid T > t)}{\Delta t}, \tag{5-10}$$

$$\lambda(t) = \frac{f(t)}{1 - F(t)} = -\frac{\mathrm{d}\ln R(t)}{\mathrm{d}t}. \tag{5-11}$$

Jej przebieg zależy od procesów zachodzących w elementach: adaptacja w początkowym okresie użytkowania powoduje spadek intensywności, a zużycie – jej wzrost. Kształt $\lambda(t)$ jest podstawą doboru modelu probabilistycznego trwałości.

Empiryczną intensywność uszkodzeń wyznacza się, odnosząc liczbę uszkodzeń w przedziale do liczby elementów *zdatnych na początku tego przedziału*:

$$\lambda^*(t) = \frac{\Delta m(t, \Delta t)}{n(t)\,\Delta t}, \tag{5-12}$$

czyli dla danych rejestrowanych w chwilach t_i (przykład 5.4 w [1]):

$$\lambda^*(t_i) = \frac{m(t_i)}{\left[n(0) - \sum_{j=1}^{i-1} m(t_j)\right](t_i - t_{i-1})}.$$

Uwagi.

- W książce symbol $n(t_i)$ ma dwa znaczenia: we wzorze (5-5) jest to liczba elementów zdatnych *po* uwzględnieniu uszkodzeń zarejestrowanych w chwili t_i, a w przykładzie 5.4 – liczba zdatnych *przed* nimi. W programie do wyznaczenia λ^* użyto drugiego znaczenia, bo tylko ono odtwarza wyniki z książki.
- Jeżeli na początku przedziału nie ma już elementów zdatnych, mianownik jest równy zeru i λ^* jest nieokreślona. Program pokazuje wtedy znak „—” (w książce tablica z wynikami jest w tym miejscu ucięta).

1.7 Wartość oczekiwana trwałości

Podstawowymi parametrami rozkładu trwałości są wartość oczekiwana i wariancja:

$$ET = \int_0^\infty R(t)\,\mathrm{d}t = \int_0^\infty t\,f(t)\,\mathrm{d}t, \tag{5-13}$$

$$VT = 2\int_0^\infty t\,R(t)\,\mathrm{d}t - [ET]^2. \tag{5-14}$$

Empiryczną wartość oczekiwaną wyznacza się z zaobserwowanych trwałości t_k poszczególnych sztuk w próbce o liczności n:

$$E^*T = \frac{1}{n}\sum_{k=1}^{n} t_k. \tag{5-15}$$

Program podaje dwie wartości E^*T:

1. **Ze wzoru (5-15).** Gdy znane są pojedyncze czasy uszkodzeń, jest to ich średnia arytmetyczna. Gdy dane mają postać tabeli $(t_j, m(t_j))$, pojedyncze trwałości nie są znane, więc za trwałość elementu przyjmuje się chwilę rejestracji jego uszkodzenia:

   $$E^*T = \frac{1}{n(0)}\sum_{j=1}^{k} t_j\,m(t_j).$$

   Wynik jest wtedy zawyżony, bo element uszkodził się nie później niż w chwili t_j.

2. **Z empirycznej funkcji niezawodności.** Książka wspomina (s. 213), że E^*T można wyznaczyć także z empirycznej dystrybuanty lub funkcji niezawodności. W programie zrealizowano to, całkując $R^*(t)$ zgodnie ze wzorem (5-13) metodą trapezów:

   $$E^*T \approx \sum_{i=1}^{k} \frac{R^*(t_{i-1}) + R^*(t_i)}{2}\,\Delta t_i, \qquad R^*(t_0) = 1.$$

Jeżeli badanie zakończono, zanim uszkodziły się wszystkie elementy, trwałości pozostałych sztuk nie są znane. Wzoru (5-15) nie można wtedy zastosować, a całka z $R^*(t)$ do chwili zakończenia obserwacji jest oszacowaniem E^*T od dołu.

1.8 Próba losowa

Zamiast danych z rzeczywistego badania program może wylosować próbę: n czasów do uszkodzenia z wybranego rozkładu trwałości. Pozwala to porównać wskaźniki empiryczne z teoretycznymi. Dostępne są trzy rozkłady (tab. 1).

Tabela 1: Rozkłady trwałości dostępne przy losowaniu próby

Rozkład	$F(t)$	$\lambda(t)$	ET
wykładniczy (λ)	$1 - e^{-\lambda t}$	λ	$\dfrac{1}{\lambda}$
Weibulla (β, η)	$1 - \exp\left[-\left(\dfrac{t}{\eta}\right)^{\beta}\right]$	$\dfrac{\beta}{\eta}\left(\dfrac{t}{\eta}\right)^{\beta-1}$	$\eta\,\Gamma\left(1+\dfrac{1}{\beta}\right)$
normalny obcięty (μ, σ)	$\dfrac{\Phi\left(\frac{t-\mu}{\sigma}\right) - \Phi\left(-\frac{\mu}{\sigma}\right)}{\Phi\left(\frac{\mu}{\sigma}\right)}$	$\dfrac{f(t)}{1-F(t)}$	$\mu + \sigma\,\dfrac{\varphi\left(\frac{\mu}{\sigma}\right)}{\Phi\left(\frac{\mu}{\sigma}\right)}$

Rozkład wykładniczy opisuje elementy o stałej intensywności uszkodzeń. Rozkład Weibulla dla $\beta > 1$ daje intensywność rosnącą (zużycie), a dla $\beta < 1$ – malejącą (adaptacja). Rozkład normalny obcięto do wartości dodatnich, ponieważ trwałość nie może być ujemna; Φ i φ oznaczają dystrybuantę i gęstość standardowego rozkładu normalnego.

Czasy losuje się z liczb pseudolosowych generatora PCG z biblioteki standardowej języka Go [2]. Dla rozkładu wykładniczego $T = E/\lambda$, a dla rozkładu Weibulla $T = \eta\,E^{1/\beta}$, gdzie E jest liczbą z rozkładu wykładniczego o parametrze 1. Dla rozkładu normalnego $T = \mu + \sigma Z$, gdzie Z pochodzi ze standardowego rozkładu normalnego, a losowanie powtarza się, dopóki wynik nie jest dodatni. Generator jest inicjowany ziarnem, więc to samo ziarno zawsze daje tę samą próbę.

1.9 Grupowanie pojedynczych czasów uszkodzeń

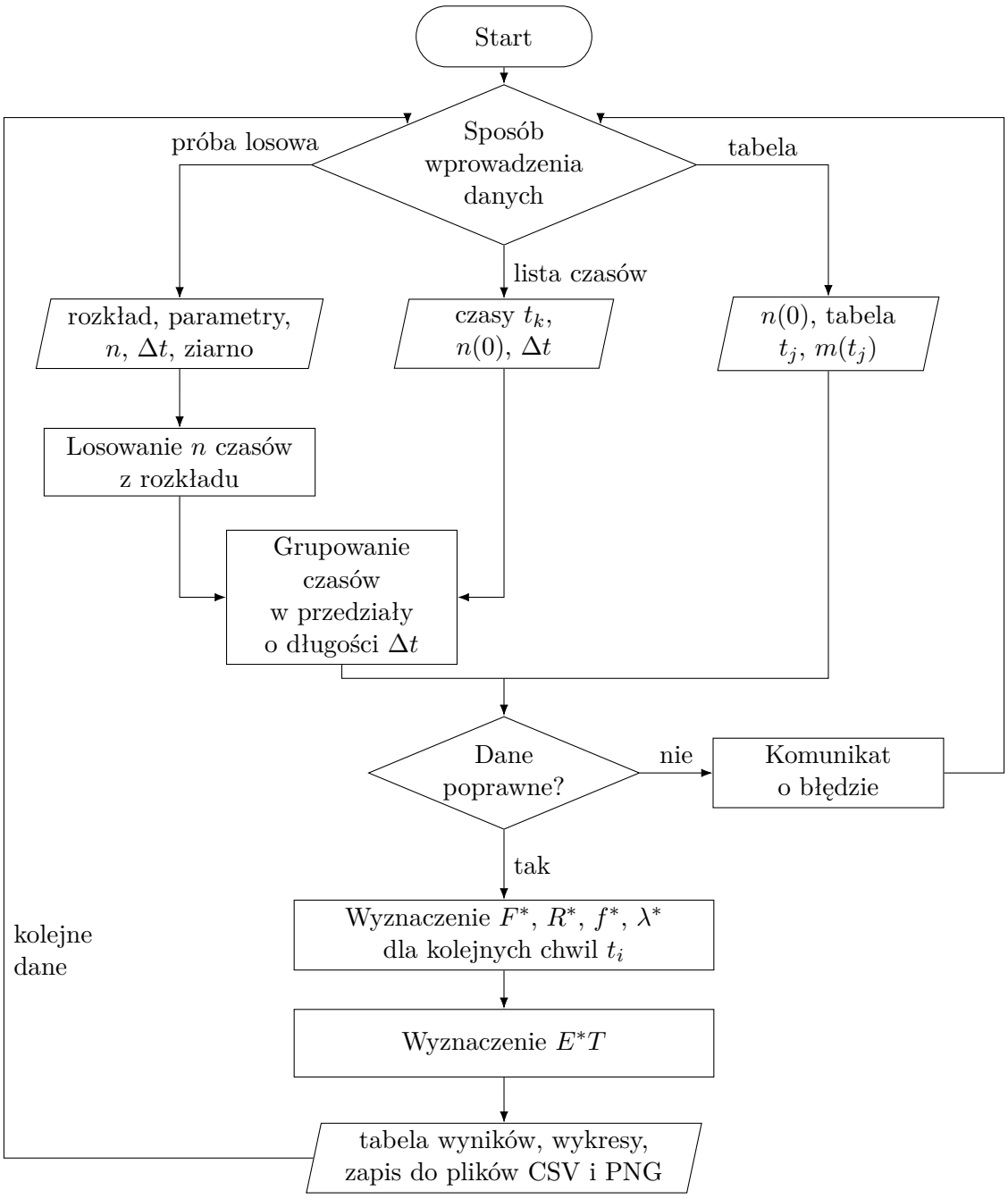
Wzory (5-2)–(5-12) wymagają liczby uszkodzeń w przedziałach czasu. Pojedyncze czasy uszkodzeń – wylosowane albo wpisane przez użytkownika – trzeba więc najpierw pogrupować w przedziały $(t_{i-1}, t_i]$ o jednakowej długości Δt, gdzie $t_i = i\,\Delta t$. Czas t trafia do przedziału o numerze $i = \lceil t/\Delta t \rceil$, a liczba przedziałów wynosi $k = \lceil t_{\max}/\Delta t \rceil$.

Długość Δt podaje użytkownik albo dobiera ją program. W drugim przypadku liczbę przedziałów przyjmuje się równą $k \approx \sqrt{n}$ (nie mniej niż 5 i nie więcej niż 50), a wartość $t_{\max}/k$ zaokrągla się w górę do najbliższej liczby postaci 1, 2, 2,5 lub 5 razy potęga dziesięciu, tak aby granice przedziałów były „okrągłe”.

2 Algorytm działania programu

2.1 Ogólny przebieg

Program przyjmuje dane na trzy sposoby: losuje próbę z zadanego rozkładu, wczytuje listę pojedynczych czasów uszkodzeń albo wczytuje tabelę uszkodzeń $(t_j, m(t_j))$. Niezależnie od sposobu dane sprowadza się do tej samej postaci – liczności próby $n(0)$ i liczb uszkodzeń w kolejnych przedziałach – po czym wyznacza się wskaźniki i prezentuje wyniki (rys. 1).



Rysunek 1: Schemat działania programu

2.2 Wyznaczanie wskaźników

Dane wejściowe: liczność próby $n(0)$ oraz ciąg par $(t_j, m(t_j))$, $j = 1, \ldots, k$.

1. **Sprawdzenie danych.** Liczność $n(0)$ musi być dodatnia, ciąg nie może być pusty, chwile t_j muszą być dodatnie i rosnące, liczby $m(t_j)$ nieujemne, a łączna liczba uszkodzeń nie może przekroczyć $n(0)$. Błąd jest zgłaszany wraz z numerem pozycji, na której wystąpił.
2. **Inicjalizacja.** $t_0 \leftarrow 0$, liczba elementów zdatnych $z \leftarrow n(0)$.
3. **Dla każdego** $i = 1, \ldots, k$**:**
   (a) $\Delta t_i \leftarrow t_i - t_{i-1}$;
   (b) $f^*(t_i) \leftarrow m(t_i)/(n(0)\,\Delta t_i)$;
   (c) jeżeli $z > 0$, to $\lambda^*(t_i) \leftarrow m(t_i)/(z\,\Delta t_i)$, w przeciwnym razie $\lambda^*(t_i)$ jest nieokreślona;
   (d) $z \leftarrow z - m(t_i)$, czyli $n(t_i) = z$;
   (e) $F^*(t_i) \leftarrow (n(0) - z)/n(0), \quad R^*(t_i) \leftarrow z/n(0)$.
4. **Wartość oczekiwana z** $R^*(t)$**:** $\sum_{i=1}^{k} \frac{1}{2}\,[R^*(t_{i-1}) + R^*(t_i)]\,\Delta t_i$, gdzie $R^*(t_0) = 1$.
5. **Wartość oczekiwana ze wzoru (5-15):** jeżeli po zakończeniu obserwacji pozostały elementy zdatne ($z > 0$), wartość jest nieokreślona; w przeciwnym razie jest to średnia pojedynczych czasów uszkodzeń, a gdy nie są one znane – $\frac{1}{n(0)} \sum_j t_j\, m(t_j)$.

Złożoność obliczeniowa jest liniowa względem liczby przedziałów k.

2.3 Grupowanie czasów uszkodzeń

Dane wejściowe: czasy $t^{(1)}, \ldots, t^{(n)}$, liczność próby $n(0) \geq n$ oraz długość przedziału Δt (podana lub dobierana automatycznie, p. 1.9).

1. Sprawdzenie danych: lista nie może być pusta, wszystkie czasy muszą być dodatnie, $\Delta t > 0$, a liczba przedziałów nie może przekroczyć 10 000.
2. $k \leftarrow \lceil t_{\max}/\Delta t \rceil$; utworzenie k przedziałów o końcach $t_i = i\,\Delta t$ i licznikach $m(t_i) = 0$.
3. Dla każdego czasu $t^{(l)}$: $i \leftarrow \lceil t^{(l)}/\Delta t \rceil$, $m(t_i) \leftarrow m(t_i) + 1$.
4. Wyznaczenie wskaźników jak w p. 2.2. Jeżeli $n(0) > n$, pozostałe $n(0) - n$ elementów traktuje się jako zdatne do końca obserwacji.

Przy wyznaczaniu numeru przedziału od ilorazu $t/\Delta t$ odejmuje się małą tolerancję (10^{-9}). Dzięki temu czas leżący dokładnie na granicy przedziału trafia do przedziału niższego także wtedy, gdy błąd zaokrąglenia liczb zmiennoprzecinkowych podnosi iloraz nieznacznie ponad liczbę całkowitą (np. 0,3/0,1).

2.4 Losowanie próby

1. Sprawdzenie parametrów rozkładu (wszystkie muszą być dodatnie) i liczności próby ($1 \leq n \leq 1\,000\,000$).
2. Zainicjowanie generatora liczb pseudolosowych ziarnem podanym przez użytkownika; gdy pole jest puste, ziarnem jest bieżący czas systemowy. Użyte ziarno jest wyświetlane w podsumowaniu, aby próbę dało się odtworzyć.
3. Wylosowanie n czasów zgodnie z p. 1.8.

4. Pogrupowanie czasów i wyznaczenie wskaźników; $n(0) = n$.
5. Zapamiętanie rozkładu teoretycznego, aby nanieść jego przebieg na wykresy.

3 Kod programu

Program napisano w języku Go [2] z wykorzystaniem bibliotek Fyne [3] (okno programu) oraz Gonum Plot [4] (wykresy). Poniżej zamieszczono pełny kod źródłowy programu. Nazwy i komentarze w kodzie są w języku angielskim, a teksty widoczne dla użytkownika – w języku polskim. Kod testów automatycznych pominięto; jest dostarczany razem z plikami programu.

3.1 Uruchomienie programu

main.go

```go
// Command empirical-reliability computes the empirical reliability indicators
// F*(t), R*(t), f*(t), λ*(t) and E*T from a random sample or from manually
// entered data.
package main

import "empirical-reliability/internal/ui"

func main() {
    ui.Run()
}
```

3.2 Wyznaczanie wskaźników

internal/reliability/indicators.go

```go
// Package reliability computes the empirical reliability indicators F*(t),
// R*(t), f*(t), λ*(t) and E*T as defined in: Niezawodność urządzeń
// elektronicznych, ed. M. Prażewska, WKiŁ, Warszawa 1987, pp. 208-213.
// Equation numbers in comments refer to that book.
package reliability

import "math"

// initialReliability is R(0): every element works at t = 0.
const initialReliability = 1.0

// Observation is one inspection: Failures elements were found failed at Time,
// i.e. they failed in the interval (previous inspection, Time].
type Observation struct {
    Time     float64
    Failures int
}

// Row holds the indicators estimated for one inspection time t_i.
type Row struct {
    Time        float64 // t_i
    Step        float64 // Δt = t_i - t_{i-1}
    Failures    int     // m(t_i)
    Working     int     // n(t_i), elements still working at t_i
    CDF         float64 // F*(t_i), eq. (5-2)
    Reliability float64 // R*(t_i), eq. (5-5)
    Density     float64 // f*(t_i), eq. (5-9)
    Hazard      float64 // λ*(t_i), eq. (5-12); NaN when no element works at the start of the interval
}

// Result is the complete set of indicators for one sample.
type Result struct {
    SampleSize int // n(0)
    Failed     int
    Survived   int // elements still working when the observation ended
    Rows       []Row

    // Mean is E*T from eq. (5-15). Grouped data carry no individual lifetimes,
    // so the inspection time is used as the lifetime of every element found
```

```
    // failed at it. NaN when some elements survived: their lifetimes are unknown.
    Mean float64
    // MeanFromReliability is E*T from eq. (5-13), integrating R*(t) with the
    // trapezoidal rule. It is a lower bound when some elements survived.
    MeanFromReliability float64

    // Times holds the individual failure times in ascending order; empty for
    // grouped data.
    Times []float64
}

// Compute estimates the indicators from grouped data: the sample size n(0) and
// the failure counts found at consecutive inspection times.
func Compute(sampleSize int, observations []Observation) (*Result, error) {
    if err := validateObservations(sampleSize, observations); err != nil {
        return nil, err
    }
    rows := buildRows(sampleSize, observations)
    survived := rows[len(rows)-1].Working
    return &Result{
        SampleSize:          sampleSize,
        Failed:              sampleSize - survived,
        Survived:            survived,
        Rows:                rows,
        Mean:                meanOfInspectionTimes(sampleSize, survived, observations),
        MeanFromReliability: integrateReliability(rows),
    }, nil
}

func validateObservations(sampleSize int, observations []Observation) error {
    if sampleSize <= 0 {
        return ErrSampleSizeNotPositive
    }
    if len(observations) == 0 {
        return ErrNoObservations
    }
    previousTime, working := 0.0, sampleSize
    for i, o := range observations {
        if err := validateObservation(o, previousTime, working); err != nil {
            return &ItemError{Position: i + 1, Err: err}
        }
        previousTime, working = o.Time, working-o.Failures
    }
    return nil
}

func validateObservation(o Observation, previousTime float64, working int) error {
    switch {
    case !isFinite(o.Time) || o.Time <= previousTime:
        return ErrTimeNotIncreasing
    case o.Failures < 0:
        return ErrNegativeFailures
    case o.Failures > working:
        return ErrFailuresExceedSample
    }
    return nil
}

func buildRows(sampleSize int, observations []Observation) []Row {
    rows := make([]Row, 0, len(observations))
    previousTime, working := 0.0, sampleSize
    for _, o := range observations {
        row := newRow(o, o.Time-previousTime, working, sampleSize)
        rows = append(rows, row)
        previousTime, working = o.Time, row.Working
    }
    return rows
}

// newRow estimates the indicators for one interval of length step that starts
// with workingAtStart working elements.
func newRow(o Observation, step float64, workingAtStart, sampleSize int) Row {
    working := workingAtStart - o.Failures
```

```
    return Row{
        Time:        o.Time,
        Step:        step,
        Failures:    o.Failures,
        Working:     working,
        CDF:         float64(sampleSize-working) / float64(sampleSize),
        Reliability: float64(working) / float64(sampleSize),
        Density:     float64(o.Failures) / (float64(sampleSize) * step),
        Hazard:      hazard(o.Failures, workingAtStart, step),
    }
}

func hazard(failures, workingAtStart int, step float64) float64 {
    if workingAtStart == 0 {
        return math.NaN()
    }
    return float64(failures) / (float64(workingAtStart) * step)
}

func meanOfInspectionTimes(sampleSize, survived int, observations []Observation) float64 {
    if survived > 0 {
        return math.NaN()
    }
    total := 0.0
    for _, o := range observations {
        total += o.Time * float64(o.Failures)
    }
    return total / float64(sampleSize)
}

func integrateReliability(rows []Row) float64 {
    area, previous := 0.0, initialReliability
    for _, row := range rows {
        area += (previous + row.Reliability) / 2 * row.Step
        previous = row.Reliability
    }
    return area
}

func isFinite(value float64) bool {
    return !math.IsNaN(value) && !math.IsInf(value, 0)
}
```

internal/reliability/grouping.go

```
package reliability

import (
    "math"
    "slices"
)

// MaxIntervals limits the number of intervals failure times may be grouped into.
const MaxIntervals = 10000

const (
    minAutoIntervals = 5
    maxAutoIntervals = 50

    // boundaryTolerance keeps a time lying on an interval boundary in the
    // lower interval despite floating-point rounding.
    boundaryTolerance = 1e-9
)

// roundMultipliers are the leading digits a rounded interval length may have.
var roundMultipliers = []float64{1, 2, 2.5, 5, 10}

// StepSelector chooses the interval length Δt for grouping failure times.
type StepSelector func(times []float64) float64

// FixedStep returns a selector that always chooses the given interval length.
func FixedStep(step float64) StepSelector {
```

```
28      return func([]float64) float64 { return step }
29  }
30
31  // AutoStep chooses the interval length from the data: about √n intervals,
32  // with the length rounded up to a round number (1, 2, 2.5 or 5 times a power
33  // of ten). It is a StepSelector.
34  func AutoStep(times []float64) float64 {
35      if len(times) == 0 {
36          return math.NaN()
37      }
38      intervals := math.Ceil(math.Sqrt(float64(len(times))))
39      intervals = min(max(intervals, minAutoIntervals), maxAutoIntervals)
40      return roundUp(slices.Max(times) / intervals)
41  }
42
43  func roundUp(value float64) float64 {
44      magnitude := math.Pow(10, math.Floor(math.Log10(value)))
45      for _, multiplier := range roundMultipliers {
46          if rounded := multiplier * magnitude; rounded >= value*(1-boundaryTolerance) {
47              return rounded
48          }
49      }
50      return value
51  }
52
53  // Group counts failure times in the intervals (t_{i-1}, t_i], where t_i = i·step.
54  func Group(times []float64, step float64) ([]Observation, error) {
55      if err := validateTimes(times); err != nil {
56          return nil, err
57      }
58      return groupValidated(times, step)
59  }
60
61  func groupValidated(times []float64, step float64) ([]Observation, error) {
62      if !isFinite(step) || step <= 0 {
63          return nil, ErrStepNotPositive
64      }
65      longest := slices.Max(times)
66      if longest/step > MaxIntervals {
67          return nil, ErrTooManyIntervals
68      }
69
70      observations := make([]Observation, intervalIndex(longest, step))
71      for i := range observations {
72          observations[i].Time = float64(i+1) * step
73      }
74      for _, t := range times {
75          observations[intervalIndex(t, step)-1].Failures++
76      }
77      return observations, nil
78  }
79
80  func validateTimes(times []float64) error {
81      if len(times) == 0 {
82          return ErrNoFailureTimes
83      }
84      for i, t := range times {
85          if !isFinite(t) || t <= 0 {
86              return &ItemError{Position: i + 1, Err: ErrFailureTimeNotPositive}
87          }
88      }
89      return nil
90  }
91
92  // intervalIndex returns the 1-based index i of the interval (t_{i-1}, t_i]
93  // that contains t.
94  func intervalIndex(t, step float64) int {
95      return max(int(math.Ceil(t/step-boundaryTolerance)), 1)
96  }
```

internal/reliability/sample.go

```
package reliability

import "slices"

// FromTimes estimates the indicators from individual failure times, grouping
// them into intervals whose length is chosen by selectStep. When sampleSize
// exceeds the number of times, the remaining elements are treated as still
// working at the end of the observation.
func FromTimes(times []float64, sampleSize int, selectStep StepSelector) (*Result, error) {
    if err := validateTimes(times); err != nil {
        return nil, err
    }
    if sampleSize < len(times) {
        return nil, ErrSampleSmallerThanTimes
    }
    observations, err := groupValidated(times, selectStep(times))
    if err != nil {
        return nil, err
    }
    result, err := Compute(sampleSize, observations)
    if err != nil {
        return nil, err
    }

    result.Times = slices.Sorted(slices.Values(times))
    if result.Survived == 0 {
        result.Mean = mean(times)
    }
    return result, nil
}

func mean(values []float64) float64 {
    total := 0.0
    for _, v := range values {
        total += v
    }
    return total / float64(len(values))
}
```

internal/reliability/errors.go

```
package reliability

import (
    "errors"
    "fmt"
)

// Validation errors reported for malformed input data.
var (
    ErrSampleSizeNotPositive  = errors.New("sample size must be positive")
    ErrNoObservations         = errors.New("no observations")
    ErrTimeNotIncreasing      = errors.New("observation times must be positive and strictly
     increasing")
    ErrNegativeFailures       = errors.New("failure count must not be negative")
    ErrFailuresExceedSample   = errors.New("failures exceed the sample size")
    ErrNoFailureTimes         = errors.New("no failure times")
    ErrFailureTimeNotPositive = errors.New("failure time must be positive")
    ErrStepNotPositive        = errors.New("interval length must be positive")
    ErrTooManyIntervals       = errors.New("interval length yields too many intervals")
    ErrSampleSmallerThanTimes = errors.New("sample size is smaller than the number of failure times")
)

// ItemError locates an error at a 1-based position in a sequence of input values.
type ItemError struct {
    Position int
    Err      error
}

func (e *ItemError) Error() string { return fmt.Sprintf("item %d: %v", e.Position, e.Err) }
func (e *ItemError) Unwrap() error { return e.Err }
```

3.3 Rozkłady trwałości

internal/distribution/distribution.go

```go
// Package distribution provides the theoretical lifetime distributions a
// random sample can be drawn from and compared against.
package distribution

import (
    "errors"
    "fmt"
    "math"
    "math/rand/v2"
)

// MaxSampleSize is the largest random sample that can be drawn.
const MaxSampleSize = 1_000_000

// negligibleSurvival is the reliability below which the hazard rate is no
// longer computed, because the quotient becomes numerically meaningless.
const negligibleSurvival = 1e-12

var (
    ErrSampleSizeNotPositive = errors.New("sample size must be positive")
    ErrSampleSizeTooLarge    = errors.New("sample size is too large")
    ErrParameterCount        = errors.New("wrong number of distribution parameters")
)

// ParameterError reports a distribution parameter that is not positive.
type ParameterError struct {
    Symbol string
}

func (e *ParameterError) Error() string {
    return fmt.Sprintf("parameter %s must be positive", e.Symbol)
}

// Sampler draws lifetimes.
type Sampler interface {
    Sample(rng *rand.Rand) float64
}

// Model describes the theoretical shape of a lifetime distribution.
type Model interface {
    CDF(t float64) float64 // F(t)
    PDF(t float64) float64 // f(t)
}

// Distribution is a lifetime distribution of the random variable T.
type Distribution interface {
    Sampler
    Model
    Mean() float64 // ET
}

// Hazard returns the theoretical hazard rate λ(t) = f(t) / (1 - F(t)), or NaN
// where the reliability is negligible.
func Hazard(m Model, t float64) float64 {
    survival := 1 - m.CDF(t)
    if survival < negligibleSurvival {
        return math.NaN()
    }
    return m.PDF(t) / survival
}

// Sample draws size lifetimes from s.
func Sample(s Sampler, size int, rng *rand.Rand) ([]float64, error) {
    if size <= 0 {
        return nil, ErrSampleSizeNotPositive
    }
    if size > MaxSampleSize {
        return nil, ErrSampleSizeTooLarge
```

```
    }
    times := make([]float64, size)
    for i := range times {
        times[i] = s.Sample(rng)
    }
    return times, nil
}

// drawPositive repeats draw until it yields a positive value, because a
// lifetime of zero is not admissible.
func drawPositive(draw func() float64) float64 {
    for {
        if value := draw(); value > 0 {
            return value
        }
    }
}

func isPositive(value float64) bool {
    return value > 0 && !math.IsInf(value, 0)
}
```

internal/distribution/catalog.go

```
package distribution

// Parameter describes one parameter of a distribution family.
type Parameter struct {
    Symbol  string
    Default float64
}

// Spec describes a distribution family: its identifier, its parameters and
// how to build a distribution from parameter values. Supporting a new family
// only takes a new Spec in the catalog.
type Spec struct {
    ID         string
    Parameters []Parameter
    build      func(values []float64) Distribution
}

// New builds a distribution from one value per parameter, in the order of
// Parameters. Every parameter must be positive.
func (s Spec) New(values ...float64) (Distribution, error) {
    if len(values) != len(s.Parameters) {
        return nil, ErrParameterCount
    }
    for i, value := range values {
        if !isPositive(value) {
            return nil, &ParameterError{Symbol: s.Parameters[i].Symbol}
        }
    }
    return s.build(values), nil
}

// The distribution families available to the application.
var (
    Exponential = Spec{
        ID:         "exponential",
        Parameters: []Parameter{{Symbol: "λ", Default: 0.01}},
        build:      func(v []float64) Distribution { return exponential{rate: v[0]} },
    }
    Weibull = Spec{
        ID:         "weibull",
        Parameters: []Parameter{{Symbol: "β", Default: 2}, {Symbol: "η", Default: 100}},
        build:      func(v []float64) Distribution { return weibull{shape: v[0], scale: v[1]} },
    }
    Normal = Spec{
        ID:         "normal",
        Parameters: []Parameter{{Symbol: "μ", Default: 100}, {Symbol: "σ", Default: 20}},
        build:      func(v []float64) Distribution { return newNormal(v[0], v[1]) },
    }
```

```
)

// Catalog lists the available distribution families.
func Catalog() []Spec {
    return []Spec{Exponential, Weibull, Normal}
}
```

internal/distribution/exponential.go

```
package distribution

import (
    "math"
    "math/rand/v2"
)

// exponential is the exponential distribution with a constant hazard rate.
type exponential struct {
    rate float64 // λ
}

func (d exponential) Sample(rng *rand.Rand) float64 {
    return drawPositive(rng.ExpFloat64) / d.rate
}

func (d exponential) Mean() float64 { return 1 / d.rate }

func (d exponential) CDF(t float64) float64 {
    if t <= 0 {
        return 0
    }
    return 1 - math.Exp(-d.rate*t)
}

func (d exponential) PDF(t float64) float64 {
    if t < 0 {
        return 0
    }
    return d.rate * math.Exp(-d.rate*t)
}
```

internal/distribution/weibull.go

```
package distribution

import (
    "math"
    "math/rand/v2"
)

// weibull is the two-parameter Weibull distribution.
type weibull struct {
    shape float64 // β
    scale float64 // η
}

func (d weibull) Sample(rng *rand.Rand) float64 {
    return d.scale * math.Pow(drawPositive(rng.ExpFloat64), 1/d.shape)
}

func (d weibull) Mean() float64 { return d.scale * math.Gamma(1+1/d.shape) }

func (d weibull) CDF(t float64) float64 {
    if t <= 0 {
        return 0
    }
    return 1 - math.Exp(-math.Pow(t/d.scale, d.shape))
}

func (d weibull) PDF(t float64) float64 {
    if t < 0 {
```

```
        return 0
    }
    x := t / d.scale
    return d.shape / d.scale * math.Pow(x, d.shape-1) * math.Exp(-math.Pow(x, d.shape))
}
```

internal/distribution/normal.go

```
package distribution

import (
    "math"
    "math/rand/v2"
)

// normal is the normal distribution truncated to positive values, because a
// lifetime cannot be negative.
type normal struct {
    mean      float64 // μ of the untruncated distribution
    deviation float64 // σ of the untruncated distribution
    mass      float64 // probability mass left after the truncation
}

func newNormal(mean, deviation float64) normal {
    return normal{
        mean:      mean,
        deviation: deviation,
        mass:      1 - standardCDF(-mean/deviation),
    }
}

func (d normal) Sample(rng *rand.Rand) float64 {
    return drawPositive(func() float64 { return d.mean + d.deviation*rng.NormFloat64() })
}

func (d normal) Mean() float64 {
    return d.mean + d.deviation*standardPDF(-d.mean/d.deviation)/d.mass
}

func (d normal) CDF(t float64) float64 {
    if t <= 0 {
        return 0
    }
    return (standardCDF((t-d.mean)/d.deviation) - (1 - d.mass)) / d.mass
}

func (d normal) PDF(t float64) float64 {
    if t < 0 {
        return 0
    }
    return standardPDF((t-d.mean)/d.deviation) / (d.deviation * d.mass)
}

func standardCDF(x float64) float64 { return 0.5 * math.Erfc(-x/math.Sqrt2) }
func standardPDF(x float64) float64 { return math.Exp(-x*x/2) / math.Sqrt(2*math.Pi) }
```

3.4 Obsługa trzech sposobów wprowadzania danych

internal/analysis/analysis.go

```
// Package analysis holds the application's use cases: it turns each kind of
// input data into reliability indicators ready to be presented.
package analysis

import (
    "math/rand/v2"
    "time"

    "empirical-reliability/internal/distribution"
    "empirical-reliability/internal/reliability"
)
```

```

// seedStreamMix derives the second word of the generator state from the seed,
// so that a single number reproduces a sample.
const seedStreamMix = 0x9e3779b97f4a7c15

// Analysis is the outcome of one calculation.
type Analysis struct {
    Result *reliability.Result
    // Random describes the drawn sample; nil for manually entered data.
    Random *RandomSample
}

// RandomSample records how a random sample was drawn.
type RandomSample struct {
    Spec       distribution.Spec
    Parameters []float64
    Seed       uint64
    Model      distribution.Distribution
}

// RandomRequest describes a random sample to draw and analyse.
type RandomRequest struct {
    Spec       distribution.Spec
    Parameters []float64
    Size       int
    Seed       uint64
    Step       reliability.StepSelector
}

// Model returns the theoretical distribution the sample was drawn from, or nil
// for manually entered data.
func (a *Analysis) Model() distribution.Model {
    if a.Random == nil {
        return nil
    }
    return a.Random.Model
}

// FromTable analyses grouped data: a sample size and a failure table.
func FromTable(sampleSize int, observations []reliability.Observation) (*Analysis, error) {
    result, err := reliability.Compute(sampleSize, observations)
    if err != nil {
        return nil, err
    }
    return &Analysis{Result: result}, nil
}

// FromTimes analyses individual failure times.
func FromTimes(times []float64, sampleSize int, step reliability.StepSelector) (*Analysis, error) {
    result, err := reliability.FromTimes(times, sampleSize, step)
    if err != nil {
        return nil, err
    }
    return &Analysis{Result: result}, nil
}

// FromRandom draws a sample from the requested distribution and analyses it.
// The same request always yields the same sample.
func FromRandom(request RandomRequest) (*Analysis, error) {
    model, err := request.Spec.New(request.Parameters...)
    if err != nil {
        return nil, err
    }
    rng := rand.New(rand.NewPCG(request.Seed, request.Seed^seedStreamMix))
    times, err := distribution.Sample(model, request.Size, rng)
    if err != nil {
        return nil, err
    }
    result, err := reliability.FromTimes(times, len(times), request.Step)
    if err != nil {
        return nil, err
    }
    return &Analysis{
```

```
        Result: result,
        Random: &RandomSample{
            Spec:       request.Spec,
            Parameters: request.Parameters,
            Seed:       request.Seed,
            Model:      model,
        },
    }, nil
}

// TimeSeed returns a seed that differs between runs.
func TimeSeed() uint64 {
    return uint64(time.Now().UnixNano())
}
```

3.5 Wczytywanie danych

internal/input/input.go

```
// Package input converts text typed by the user into data for the analysis.
package input

import (
    "errors"
    "fmt"
    "strconv"
    "strings"

    "empirical-reliability/internal/numfmt"
    "empirical-reliability/internal/reliability"
)

const (
    fieldSeparators = " \t\r\n;"
    lineSeparator   = "\n"
    columnSeparator = " "
    tableColumns    = 2
)

// ErrColumnCount reports a table row that does not hold exactly a time and a
// failure count.
var ErrColumnCount = errors.New("a table row must hold a time and a failure count")

// LineError locates an error on a 1-based line of the failure table.
type LineError struct {
    Line int
    Err  error
}

func (e *LineError) Error() string { return fmt.Sprintf("line %d: %v", e.Line, e.Err) }
func (e *LineError) Unwrap() error { return e.Err }

// ParseTimes reads failure times separated by whitespace or semicolons.
func ParseTimes(text string) ([]float64, error) {
    fields := splitFields(text)
    times := make([]float64, len(fields))
    for i, field := range fields {
        value, err := numfmt.ParseFloat(field)
        if err != nil {
            return nil, &reliability.ItemError{Position: i + 1, Err: err}
        }
        times[i] = value
    }
    return times, nil
}

// ParseTable reads a failure table: every non-empty line holds an inspection
// time and the number of failures found at it.
func ParseTable(text string) ([]reliability.Observation, error) {
    var observations []reliability.Observation
    for i, line := range strings.Split(text, lineSeparator) {
        fields := splitFields(line)
```

```
        if len(fields) == 0 {
            continue
        }
        observation, err := parseRow(fields)
        if err != nil {
            return nil, &LineError{Line: i + 1, Err: err}
        }
        observations = append(observations, observation)
    }
    return observations, nil
}

// FormatTable writes observations in the form ParseTable reads.
func FormatTable(observations []reliability.Observation) string {
    lines := make([]string, len(observations))
    for i, o := range observations {
        lines[i] = numfmt.Compact(o.Time) + columnSeparator + strconv.Itoa(o.Failures)
    }
    return strings.Join(lines, lineSeparator)
}

func parseRow(fields []string) (reliability.Observation, error) {
    if len(fields) != tableColumns {
        return reliability.Observation{}, ErrColumnCount
    }
    time, err := numfmt.ParseFloat(fields[0])
    if err != nil {
        return reliability.Observation{}, err
    }
    failures, err := numfmt.ParseInt(fields[1])
    if err != nil {
        return reliability.Observation{}, err
    }
    return reliability.Observation{Time: time, Failures: failures}, nil
}

func splitFields(text string) []string {
    return strings.FieldsFunc(text, func(r rune) bool {
        return strings.ContainsRune(fieldSeparators, r)
    })
}
```

3.6 Zapis liczb

internal/numfmt/numfmt.go

```
// Package numfmt parses and formats numbers the way the user interface shows
// them: with a decimal comma. A decimal point is accepted on input as well.
package numfmt

import (
    "errors"
    "fmt"
    "math"
    "strconv"
    "strings"
)

// Undefined is shown in place of a value that cannot be determined.
const Undefined = "-"

const (
    displaySeparator = ","
    parseSeparator   = "."
    compactDigits    = 6
    floatBits        = 64
    integerBase      = 10
)

var (
    ErrNotNumber   = errors.New("not a number")
    ErrNotInteger  = errors.New("not an integer")
```

```
    ErrNotUnsigned = errors.New("not a non-negative integer")
)

// SyntaxError reports text that could not be read as a number.
type SyntaxError struct {
    Text string
    Err  error
}

func (e *SyntaxError) Error() string { return fmt.Sprintf("%q: %v", e.Text, e.Err) }
func (e *SyntaxError) Unwrap() error { return e.Err }

// ParseFloat reads a finite number written with a decimal comma or point.
func ParseFloat(text string) (float64, error) {
    text = strings.TrimSpace(text)
    value, err := strconv.ParseFloat(strings.ReplaceAll(text, displaySeparator, parseSeparator),
     floatBits)
    if err != nil || math.IsNaN(value) || math.IsInf(value, 0) {
        return 0, &SyntaxError{Text: text, Err: ErrNotNumber}
    }
    return value, nil
}

// ParseInt reads an integer.
func ParseInt(text string) (int, error) {
    text = strings.TrimSpace(text)
    value, err := strconv.Atoi(text)
    if err != nil {
        return 0, &SyntaxError{Text: text, Err: ErrNotInteger}
    }
    return value, nil
}

// ParseUint reads a non-negative integer.
func ParseUint(text string) (uint64, error) {
    text = strings.TrimSpace(text)
    value, err := strconv.ParseUint(text, integerBase, floatBits)
    if err != nil {
        return 0, &SyntaxError{Text: text, Err: ErrNotUnsigned}
    }
    return value, nil
}

// Fixed formats value with the given number of decimal places.
func Fixed(value float64, decimals int) string {
    if !isFinite(value) {
        return Undefined
    }
    return Localize(strconv.FormatFloat(value, 'f', decimals, floatBits))
}

// Compact formats value without trailing zeros, using at most six significant
// digits.
func Compact(value float64) string {
    if !isFinite(value) {
        return Undefined
    }
    return Localize(strconv.FormatFloat(value, 'g', compactDigits, floatBits))
}

// Localize replaces the decimal point of an already formatted number with the
// decimal comma.
func Localize(number string) string {
    return strings.ReplaceAll(number, parseSeparator, displaySeparator)
}

func isFinite(value float64) bool {
    return !math.IsNaN(value) && !math.IsInf(value, 0)
}
```

3.7 Wykresy

internal/chart/chart.go

```go
// Package chart draws the empirical reliability indicators, optionally
// together with the theoretical curve they estimate.
package chart

import (
    "image"
    "math"

    "gonum.org/v1/plot"
    "gonum.org/v1/plot/plotter"
    "gonum.org/v1/plot/vg/draw"
    "gonum.org/v1/plot/vg/vgimg"

    "empirical-reliability/internal/distribution"
    "empirical-reliability/internal/numfmt"
    "empirical-reliability/internal/reliability"
)

// Labels holds the texts shown on a chart.
type Labels struct {
    Title       string
    TimeAxis    string
    Empirical   string
    Theoretical string
}

// Render draws one indicator of result. When model is not nil, its curve is
// drawn as a dashed line for comparison.
func Render(indicator Indicator, result *reliability.Result, model distribution.Model, labels Labels)
     (image.Image, error) {
    definition := definitions[indicator]
    points := definition.points(result.Rows)
    lastTime := result.Rows[len(result.Rows)-1].Time

    p := newPlot(labels, lastTime, definition.upperBound(points)*headroom)
    line, err := addEmpirical(p, points, definition.interval)
    if err != nil {
        return nil, err
    }
    if model != nil {
        curve := addTheory(p, func(t float64) float64 { return definition.theory(model, t) })
        addLegend(p, labels, line, curve, definition.legendLeft)
    }
    return rasterize(p), nil
}

func newPlot(labels Labels, timeMax, valueMax float64) *plot.Plot {
    p := plot.New()
    p.Title.Text = labels.Title
    p.Title.Padding = titlePadding
    p.X.Label.Text = labels.TimeAxis
    p.X.Tick.Marker = roundTicks{}
    p.Y.Tick.Marker = roundTicks{}
    p.X.Min, p.X.Max = 0, timeMax
    p.Y.Min, p.Y.Max = 0, valueMax
    p.Add(plotter.NewGrid())
    return p
}

// addEmpirical draws the empirical values: as steps for interval indicators,
// otherwise as a line through marked points.
func addEmpirical(p *plot.Plot, points plotter.XYs, interval bool) (*plotter.Line, error) {
    line, err := plotter.NewLine(points)
    if err != nil {
        return nil, err
    }
    line.Color = empiricalColor
    line.Width = empiricalWidth
    p.Add(line)

    if interval {
        line.StepStyle = plotter.PreStep
```

```
        return line, nil
    }
    if len(points) <= maxMarkers {
        markers, err := plotter.NewScatter(points)
        if err != nil {
            return nil, err
        }
        markers.GlyphStyle = draw.GlyphStyle{Color: empiricalColor, Radius: markerRadius, Shape:
 draw.CircleGlyph{}}
        p.Add(markers)
    }
    return line, nil
}

func addTheory(p *plot.Plot, theory func(float64) float64) *plotter.Function {
    offChart := p.Y.Max * offChartFactor
    curve := plotter.NewFunction(func(t float64) float64 {
        value := theory(t)
        if math.IsNaN(value) || value > offChart {
            return offChart
        }
        return value
    })
    curve.Samples = theorySamples
    curve.Color = theoryColor
    curve.Width = theoryWidth
    curve.Dashes = theoryDashes
    p.Add(curve)
    return curve
}

func addLegend(p *plot.Plot, labels Labels, empirical, theoretical plot.Thumbnailer, left bool) {
    p.Legend.Add(labels.Empirical, empirical)
    p.Legend.Add(labels.Theoretical, theoretical)
    p.Legend.Top = true
    p.Legend.Left = left
    p.Legend.Padding = legendPadding
    p.Legend.TextStyle.Font.Size = legendFontSize
}

// rasterize draws the plot to an image. The margin keeps the outermost tick
// labels from being cut off.
func rasterize(p *plot.Plot) image.Image {
    canvas := vgimg.NewWith(vgimg.UseWH(imageWidth, imageHeight), vgimg.UseDPI(resolution))
    p.Draw(draw.Crop(draw.New(canvas), margin, -margin, margin/2, -margin/2))
    return canvas.Image()
}

// roundTicks places axis ticks at round values and labels them with a decimal
// comma.
type roundTicks struct{}

func (roundTicks) Ticks(low, high float64) []plot.Tick {
    step := tickStep(high - low)
    first := int(math.Ceil(low/step - tickTolerance))
    last := int(math.Floor(high/step + tickTolerance))
    ticks := make([]plot.Tick, 0, last-first+1)
    for i := first; i <= last; i++ {
        value := float64(i) * step
        ticks = append(ticks, plot.Tick{Value: value, Label: numfmt.Compact(value)})
    }
    return ticks
}

// tickStep returns the smallest round distance between ticks that puts at
// most targetTicks intervals on an axis of the given span.
func tickStep(span float64) float64 {
    raw := span / targetTicks
    magnitude := math.Pow(10, math.Floor(math.Log10(raw)))
    for _, multiplier := range tickMultipliers {
        if step := multiplier * magnitude; step >= raw {
            return step
        }
```

```
    }
    return raw
}
```

internal/chart/indicator.go

```
package chart

import (
    "math"

    "gonum.org/v1/plot/plotter"

    "empirical-reliability/internal/distribution"
    "empirical-reliability/internal/reliability"
)

// Indicator identifies the reliability indicator shown on a chart.
type Indicator int

const (
    CDF Indicator = iota
    Reliability
    Density
    Hazard
)

// Indicators lists the indicators that can be charted, in display order.
func Indicators() []Indicator {
    return []Indicator{CDF, Reliability, Density, Hazard}
}

// Slug returns a short identifier of the indicator, suitable for file names.
func (i Indicator) Slug() string { return definitions[i].slug }

// definition describes how one indicator is drawn. Charting a new indicator
// only takes a new entry in definitions.
type definition struct {
    slug   string
    value  func(reliability.Row) float64
    theory func(m distribution.Model, t float64) float64
    // interval marks an indicator that describes a whole interval
    // (t_{i-1}, t_i] and is therefore drawn as steps.
    interval bool
    // start is the value at t = 0 of an indicator defined at points.
    start      float64
    legendLeft bool
}

var definitions = map[Indicator]definition{
    CDF: {
        slug:       "cdf",
        value:      func(r reliability.Row) float64 { return r.CDF },
        theory:     func(m distribution.Model, t float64) float64 { return m.CDF(t) },
        start:      0,
        legendLeft: true,
    },
    Reliability: {
        slug:   "reliability",
        value:  func(r reliability.Row) float64 { return r.Reliability },
        theory: func(m distribution.Model, t float64) float64 { return 1 - m.CDF(t) },
        start:  1,
    },
    Density: {
        slug:     "density",
        value:    func(r reliability.Row) float64 { return r.Density },
        theory:   func(m distribution.Model, t float64) float64 { return m.PDF(t) },
        interval: true,
    },
    Hazard: {
        slug:       "hazard",
        value:      func(r reliability.Row) float64 { return r.Hazard },
```

```
        theory:     distribution.Hazard,
        interval:   true,
        legendLeft: true,
    },
}

// points returns the empirical values to draw, starting at t = 0 and stopping
// at the first undefined value.
func (d definition) points(rows []reliability.Row) plotter.XYs {
    start := d.start
    if d.interval {
        start = d.value(rows[0])
    }
    points := plotter.XYs{{X: 0, Y: start}}
    for _, row := range rows {
        value := d.value(row)
        if math.IsNaN(value) {
            break
        }
        points = append(points, plotter.XY{X: row.Time, Y: value})
    }
    return points
}

// upperBound returns the highest value the chart has to show.
func (d definition) upperBound(points plotter.XYs) float64 {
    if !d.interval {
        return probabilityMax
    }
    highest := 0.0
    for _, point := range points {
        highest = max(highest, point.Y)
    }
    if highest == 0 {
        return emptyRangeMax
    }
    return highest
}
```

internal/chart/style.go

```
package chart

import (
    "image/color"

    "gonum.org/v1/plot"
    "gonum.org/v1/plot/font"
    "gonum.org/v1/plot/vg"
)

const (
    imageWidth  = 6 * vg.Inch
    imageHeight = 4 * vg.Inch
    resolution  = 150 // dots per inch

    // maxMarkers is the largest number of points still drawn with markers.
    maxMarkers    = 60
    theorySamples = 400

    // headroom leaves space above the highest value.
    headroom = 1.08
    // offChartFactor places undefined theoretical values far above the chart,
    // where they are clipped away.
    offChartFactor = 10

    // targetTicks is the largest number of intervals between ticks on an axis.
    targetTicks = 6
    // tickTolerance keeps a tick lying on the end of an axis despite
    // floating-point rounding.
    tickTolerance = 1e-9

```

```
    probabilityMax = 1.0
    // emptyRangeMax is the value range of an indicator that is zero everywhere.
    emptyRangeMax = 1.0
)

var (
    margin         = vg.Points(10)
    titlePadding   = vg.Points(6)
    empiricalWidth = vg.Points(1.6)
    theoryWidth    = vg.Points(1.4)
    markerRadius   = vg.Points(2.5)
    legendPadding  = vg.Points(4)
    legendFontSize = vg.Points(10)
    theoryDashes   = []vg.Length{vg.Points(5), vg.Points(3)}

    empiricalColor = color.RGBA{R: 0x1f, G: 0x77, B: 0xb4, A: 0xff}
    theoryColor    = color.RGBA{R: 0xd6, G: 0x27, B: 0x28, A: 0xff}

    chartFont = font.Font{Typeface: "Liberation", Variant: "Sans"}

    // tickMultipliers are the leading digits a round tick distance may have.
    tickMultipliers = []float64{1, 2, 5, 10}
)

func init() {
    // gonum/plot takes the font of every text element from this package-level default.
    plot.DefaultFont = chartFont
}
```

3.8 Zapis plików

internal/export/export.go

```
// Package export writes results to files.
package export

import (
    "encoding/csv"
    "errors"
    "image"
    "image/png"
    "io"
    "os"
)

const (
    // csvSeparator is the field separator expected by spreadsheets that use a
    // decimal comma.
    csvSeparator = ';'
    // utf8BOM makes spreadsheets read the file as UTF-8.
    utf8BOM = "\xEF\xBB\xBF"
)

// WriteCSV writes records as a semicolon-separated UTF-8 file.
func WriteCSV(w io.Writer, records [][]string) error {
    if _, err := io.WriteString(w, utf8BOM); err != nil {
        return err
    }
    writer := csv.NewWriter(w)
    writer.Comma = csvSeparator
    writer.UseCRLF = true
    return writer.WriteAll(records)
}

// SavePNG writes img to a PNG file at path.
func SavePNG(path string, img image.Image) error {
    file, err := os.Create(path)
    if err != nil {
        return err
    }
    return errors.Join(png.Encode(file, img), file.Close())
}
```

3.9 Okno programu

Okno i elementy wspólne

internal/ui/app.go

```go
// Package ui is the graphical user interface of the application.
package ui

import (
    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/app"
)

// Run opens the main window and blocks until it is closed.
func Run() {
    application := app.NewWithID(appID)
    application.Settings().SetTheme(newLightTheme())

    window := application.NewWindow(windowTitle)
    window.SetContent(newMainWindow(window).content())
    window.Resize(fyne.NewSize(windowWidth, windowHeight))
    window.ShowAndRun()
}
```

internal/ui/main_window.go

```go
package ui

import (
    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/container"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/distribution"
)

// view presents one aspect of an analysis.
type view interface {
    tabbed
    show(a *analysis.Analysis)
}

// mainWindow wires the input forms to the views that present their results.
type mainWindow struct {
    host host

    random *randomForm
    table  *tableForm
    times  *timesForm

    results *resultsView
    charts  *chartsView
    sample  *sampleView

    input  *container.AppTabs
    output *container.AppTabs
}

func newMainWindow(window fyne.Window) *mainWindow {
    m := &mainWindow{host: host{window: window}}

    m.random = newRandomForm(distribution.Catalog(), m.run)
    m.table = newTableForm(m.run)
    m.times = newTimesForm(m.run)
    m.input = tabsOf(m.random, m.table, m.times)

    m.results = newResultsView(m.host)
    m.charts = newChartsView(m.host)
    m.sample = newSampleView()
    m.output = tabsOf(m.results, m.charts, m.sample)
```

```
45
46      return m
47  }
48
49  func (m *mainWindow) content() fyne.CanvasObject {
50      split := container.NewHSplit(m.input, m.output)
51      split.Offset = inputPaneShare
52      return split
53  }
54
55  func (m *mainWindow) views() []view {
56      return []view{m.results, m.charts, m.sample}
57  }
58
59  // run executes a computation requested by a form and presents its outcome.
60  func (m *mainWindow) run(c compute) {
61      a, err := c()
62      if err != nil {
63          m.host.reportError(err)
64          return
65      }
66      for _, v := range m.views() {
67          v.show(a)
68      }
69  }
```

internal/ui/host.go

```
1   package ui
2
3   import (
4       "errors"
5
6       "fyne.io/fyne/v2"
7       "fyne.io/fyne/v2/dialog"
8   )
9
10  // host gives forms and views access to the window they are shown in.
11  type host struct {
12      window fyne.Window
13  }
14
15  // reportError shows err to the user; a nil error is ignored.
16  func (h host) reportError(err error) {
17      if err != nil {
18          dialog.ShowError(errors.New(describe(err)), h.window)
19      }
20  }
21
22  func (h host) inform(title, message string) {
23      dialog.ShowInformation(title, message, h.window)
24  }
```

internal/ui/widgets.go

```
1   package ui
2
3   import (
4       "fyne.io/fyne/v2"
5       "fyne.io/fyne/v2/container"
6       "fyne.io/fyne/v2/layout"
7       "fyne.io/fyne/v2/widget"
8   )
9
10  // tabbed is a part of the window shown as a tab.
11  type tabbed interface {
12      tab() *container.TabItem
13  }
14
15  func tabsOf(parts ...tabbed) *container.AppTabs {
16      tabs := container.NewAppTabs()
```

```
    for _, part := range parts {
        tabs.Append(part.tab())
    }
    return tabs
}

// newSelect builds a select widget with an explicit placeholder.
// widget.NewSelect is avoided because it looks up a default placeholder that
// Fyne has no Polish translation for, and logs an error on every call.
func newSelect(options []string, placeholder string, changed func(string)) *widget.Select {
    selector := &widget.Select{Options: options, PlaceHolder: placeholder, OnChanged: changed}
    selector.ExtendBaseWidget(selector)
    return selector
}

func entryWithText(text string) *widget.Entry {
    entry := widget.NewEntry()
    entry.SetText(text)
    return entry
}

func entryWithPlaceholder(placeholder string) *widget.Entry {
    entry := widget.NewEntry()
    entry.SetPlaceHolder(placeholder)
    return entry
}

func multiLineEntry(placeholder string) *widget.Entry {
    entry := widget.NewMultiLineEntry()
    entry.SetPlaceHolder(placeholder)
    entry.Wrapping = fyne.TextWrapWord
    return entry
}

func wrappedLabel(text string) *widget.Label {
    label := widget.NewLabel(text)
    label.Wrapping = fyne.TextWrapWord
    return label
}

func primaryButton(label string, action func()) *widget.Button {
    button := widget.NewButton(label, action)
    button.Importance = widget.HighImportance
    return button
}

// labelled returns a label followed by the field it describes, as one row of
// a form grid.
func labelled(label string, field fyne.CanvasObject) []fyne.CanvasObject {
    return []fyne.CanvasObject{widget.NewLabel(label), field}
}

func formGrid(rows ...[]fyne.CanvasObject) *fyne.Container {
    grid := container.New(layout.NewFormLayout())
    for _, row := range rows {
        grid.Objects = append(grid.Objects, row...)
    }
    return grid
}
```

internal/ui/fields.go

```
package ui

import (
    "strings"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/numfmt"
    "empirical-reliability/internal/reliability"
)

```

```go
// compute produces an analysis from the data currently entered in a form.
type compute func() (*analysis.Analysis, error)

// runner executes a computation and presents its outcome. Forms depend on
// this abstraction instead of on the window that implements it.
type runner func(compute)

// parseField reads the text of a form field, attaching the field's label to
// the error when the text is malformed.
func parseField[T any](label, text string, parse func(string) (T, error)) (T, error) {
    value, err := parse(text)
    if err != nil {
        return value, &fieldError{label: label, err: err}
    }
    return value, nil
}

func isBlank(text string) bool {
    return strings.TrimSpace(text) == ""
}

// parseStep returns the interval length entered by the user, or the automatic
// choice when the field is left blank.
func parseStep(text string) (reliability.StepSelector, error) {
    if isBlank(text) {
        return reliability.AutoStep, nil
    }
    step, err := parseField(labelStep, text, numfmt.ParseFloat)
    if err != nil {
        return nil, err
    }
    return reliability.FixedStep(step), nil
}

// parseSeed returns the seed entered by the user, or a fresh one when the
// field is left blank.
func parseSeed(text string) (uint64, error) {
    if isBlank(text) {
        return analysis.TimeSeed(), nil
    }
    return parseField(labelSeed, text, numfmt.ParseUint)
}
```

internal/ui/theme.go

```go
package ui

import (
    "image/color"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/theme"
)

// lightTheme forces the light variant of the default theme, so that the charts
// drawn on a white background match the rest of the window regardless of the
// system settings.
type lightTheme struct {
    fyne.Theme
}

func newLightTheme() fyne.Theme {
    return lightTheme{Theme: theme.DefaultTheme()}
}

func (t lightTheme) Color(name fyne.ThemeColorName, _ fyne.ThemeVariant) color.Color {
    return t.Theme.Color(name, theme.VariantLight)
}
```

internal/ui/settings.go

```go
package ui

const (
    appID = "empirical-reliability"

    windowWidth  float32 = 1280
    windowHeight float32 = 780
    // inputPaneShare is the part of the window width taken by the input forms.
    inputPaneShare = 0.34

    // secondaryActionColumns is the number of secondary buttons shown side by side.
    secondaryActionColumns = 2

    chartColumns           = 2
    chartMinWidth  float32 = 300
    chartMinHeight float32 = 200

    // Decimal places of the displayed values.
    probabilityDecimals = 4
    rateDecimals        = 5
    meanDecimals        = 4
    exportMeanDecimals  = 6

    defaultRandomSize = "1000"

    resultsFileName    = "results.csv"
    chartFileExtension = ".png"
)
```

Formularze danych

internal/ui/random_form.go

```go
package ui

import (
    "fmt"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/widget"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/distribution"
    "empirical-reliability/internal/numfmt"
)

// randomForm collects the settings of a random sample to draw.
type randomForm struct {
    specs []distribution.Spec

    kind       *widget.Select
    parameters []*widget.Entry // one entry per parameter of the selected distribution
    size       *widget.Entry
    step       *widget.Entry
    seed       *widget.Entry
    fields     *fyne.Container
    submit     *widget.Button
    content    *fyne.Container
}

func newRandomForm(specs []distribution.Spec, run runner) *randomForm {
    f := &randomForm{
        specs:  specs,
        size:   entryWithText(defaultRandomSize),
        step:   entryWithPlaceholder(placeholderAutomatic),
        seed:   entryWithPlaceholder(placeholderRandomSeed),
        fields: formGrid(),
    }
    f.kind = newSelect(f.distributionNames(), placeholderDistribution, func(string) {
     f.showParameters() })
    f.submit = primaryButton(buttonDraw, func() { run(f.analyze) })
```

```
39      f.content = container.NewVBox(f.fields, f.submit, wrappedLabel(hintRandom))
40      f.kind.SetSelectedIndex(0)
41      return f
42  }
43
44  func (f *randomForm) tab() *container.TabItem {
45      return container.NewTabItem(tabRandom, f.content)
46  }
47
48  func (f *randomForm) distributionNames() []string {
49      names := make([]string, len(f.specs))
50      for i, spec := range f.specs {
51          names[i] = distributionNames[spec.ID]
52      }
53      return names
54  }
55
56  func (f *randomForm) selected() distribution.Spec {
57      return f.specs[f.kind.SelectedIndex()]
58  }
59
60  // showParameters rebuilds the form so that it holds one field per parameter
61  // of the selected distribution, filled with the default value.
62  func (f *randomForm) showParameters() {
63      spec := f.selected()
64      f.parameters = make([]*widget.Entry, len(spec.Parameters))
65      rows := [][]fyne.CanvasObject{labelled(labelDistribution, f.kind)}
66      for i, parameter := range spec.Parameters {
67          f.parameters[i] = entryWithText(numfmt.Compact(parameter.Default))
68          rows = append(rows, labelled(parameterLabel(parameter.Symbol), f.parameters[i]))
69      }
70      rows = append(rows,
71          labelled(labelRandomSize, f.size),
72          labelled(labelStep, f.step),
73          labelled(labelSeed, f.seed),
74      )
75      f.fields.Objects = formGrid(rows...).Objects
76      // The number of rows changes, so everything below the fields moves.
77      f.content.Refresh()
78  }
79
80  func (f *randomForm) analyze() (*analysis.Analysis, error) {
81      spec := f.selected()
82      parameters, err := f.parameterValues(spec)
83      if err != nil {
84          return nil, err
85      }
86      size, err := parseField(labelRandomSize, f.size.Text, numfmt.ParseInt)
87      if err != nil {
88          return nil, err
89      }
90      step, err := parseStep(f.step.Text)
91      if err != nil {
92          return nil, err
93      }
94      seed, err := parseSeed(f.seed.Text)
95      if err != nil {
96          return nil, err
97      }
98      return analysis.FromRandom(analysis.RandomRequest{
99          Spec:       spec,
100         Parameters: parameters,
101         Size:       size,
102         Seed:       seed,
103         Step:       step,
104     })
105 }
106
107 func (f *randomForm) parameterValues(spec distribution.Spec) ([]float64, error) {
108     values := make([]float64, len(f.parameters))
109     for i, entry := range f.parameters {
110         value, err := parseField(parameterLabel(spec.Parameters[i].Symbol), entry.Text,
      numfmt.ParseFloat)
```

```
        if err != nil {
            return nil, err
        }
        values[i] = value
    }
    return values, nil
}

func parameterLabel(symbol string) string {
    return fmt.Sprintf(parameterLabelFormat, symbol, parameterNames[symbol])
}
```

internal/ui/table_form.go

```
package ui

import (
    "strconv"

    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/widget"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/input"
    "empirical-reliability/internal/numfmt"
)

// tableForm collects grouped data: the sample size and a failure table.
type tableForm struct {
    size   *widget.Entry
    data   *widget.Entry
    submit *widget.Button
}

func newTableForm(run runner) *tableForm {
    f := &tableForm{
        size: widget.NewEntry(),
        data: widget.NewMultiLineEntry(),
    }
    f.submit = primaryButton(buttonCompute, func() { run(f.analyze) })
    f.loadExample()
    return f
}

func (f *tableForm) tab() *container.TabItem {
    top := container.NewVBox(formGrid(labelled(labelSampleSize, f.size)), wrappedLabel(hintTable))
    bottom := container.NewVBox(
        f.submit,
        container.NewGridWithColumns(secondaryActionColumns,
            widget.NewButton(buttonExample, f.loadExample),
            widget.NewButton(buttonClear, f.clear),
        ),
    )
    return container.NewTabItem(tabTable, container.NewBorder(top, bottom, nil, nil, f.data))
}

func (f *tableForm) loadExample() {
    f.size.SetText(strconv.Itoa(exampleSampleSize))
    f.data.SetText(input.FormatTable(exampleObservations))
}

func (f *tableForm) clear() {
    f.size.SetText("")
    f.data.SetText("")
}

func (f *tableForm) analyze() (*analysis.Analysis, error) {
    sampleSize, err := parseField(labelSampleSize, f.size.Text, numfmt.ParseInt)
    if err != nil {
        return nil, err
    }
    observations, err := input.ParseTable(f.data.Text)
```

```
    if err != nil {
        return nil, err
    }
    return analysis.FromTable(sampleSize, observations)
}
```

internal/ui/times_form.go

```
package ui

import (
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/widget"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/input"
    "empirical-reliability/internal/numfmt"
)

// timesForm collects individual failure times.
type timesForm struct {
    size   *widget.Entry
    step   *widget.Entry
    data   *widget.Entry
    submit *widget.Button
}

func newTimesForm(run runner) *timesForm {
    f := &timesForm{
        size: entryWithPlaceholder(placeholderTimesCount),
        step: entryWithPlaceholder(placeholderAutomatic),
        data: multiLineEntry(placeholderTimes),
    }
    f.submit = primaryButton(buttonCompute, func() { run(f.analyze) })
    return f
}

func (f *timesForm) tab() *container.TabItem {
    top := container.NewVBox(
        formGrid(labelled(labelSampleSize, f.size), labelled(labelStep, f.step)),
        wrappedLabel(hintTimes),
    )
    bottom := container.NewVBox(f.submit, widget.NewButton(buttonClear, f.clear))
    return container.NewTabItem(tabTimes, container.NewBorder(top, bottom, nil, nil, f.data))
}

func (f *timesForm) clear() {
    f.data.SetText("")
}

func (f *timesForm) analyze() (*analysis.Analysis, error) {
    times, err := input.ParseTimes(f.data.Text)
    if err != nil {
        return nil, err
    }
    sampleSize, err := f.sampleSize(len(times))
    if err != nil {
        return nil, err
    }
    step, err := parseStep(f.step.Text)
    if err != nil {
        return nil, err
    }
    return analysis.FromTimes(times, sampleSize, step)
}

// sampleSize returns the entered sample size, or the number of failure times
// when the field is left blank.
func (f *timesForm) sampleSize(timeCount int) (int, error) {
    if isBlank(f.size.Text) {
        return timeCount, nil
    }
```

```
    return parseField(labelSampleSize, f.size.Text, numfmt.ParseInt)
}
```

internal/ui/example.go

```
package ui

import "empirical-reliability/internal/reliability"

// The data of example 5.1 from the book (p. 209): 35 capacitors inspected
// every hour until the last one failed.
const exampleSampleSize = 35

var exampleObservations = []reliability.Observation{
    {Time: 1, Failures: 0},
    {Time: 2, Failures: 3},
    {Time: 3, Failures: 3},
    {Time: 4, Failures: 5},
    {Time: 5, Failures: 8},
    {Time: 6, Failures: 7},
    {Time: 7, Failures: 6},
    {Time: 8, Failures: 2},
    {Time: 9, Failures: 1},
    {Time: 10, Failures: 0},
}
```

Widoki wyników

internal/ui/results_view.go

```
package ui

import (
    "errors"
    "io"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/dialog"
    "fyne.io/fyne/v2/widget"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/export"
    "empirical-reliability/internal/reliability"
)

// resultsView shows the summary and the table of indicators.
type resultsView struct {
    host    host
    current *analysis.Analysis // nil until something is computed

    summary *widget.Label
    table   *widget.Table
    content *fyne.Container
}

func newResultsView(h host) *resultsView {
    v := &resultsView{host: h, summary: wrappedLabel(summaryPrompt)}
    v.table = v.newTable()
    save := widget.NewButton(buttonSaveCSV, v.saveCSV)
    v.content = container.NewBorder(v.summary, save, nil, nil, v.table)
    return v
}

func (v *resultsView) tab() *container.TabItem {
    return container.NewTabItem(tabResults, v.content)
}

func (v *resultsView) show(a *analysis.Analysis) {
    v.current = a
    v.summary.SetText(summaryText(a))
```

```
    v.table.Refresh()
    v.table.ScrollToTop()
    // The summary changes its height, so the table has to be laid out again.
    v.content.Refresh()
}

func (v *resultsView) newTable() *widget.Table {
    table := widget.NewTable(
        func() (int, int) { return len(v.rows()), len(resultColumns) },
        func() fyne.CanvasObject { return widget.NewLabel("") },
        func(id widget.TableCellID, cell fyne.CanvasObject) {
            row := v.rows()[id.Row]
            cell.(*widget.Label).SetText(resultColumns[id.Col].cell(id.Row, row))
        },
    )
    table.ShowHeaderRow = true
    table.CreateHeader = newHeaderCell
    table.UpdateHeader = func(id widget.TableCellID, cell fyne.CanvasObject) {
        cell.(*widget.Label).SetText(resultColumns[id.Col].header)
    }
    for i, column := range resultColumns {
        table.SetColumnWidth(i, column.width)
    }
    return table
}

func newHeaderCell() fyne.CanvasObject {
    label := widget.NewLabel("")
    label.TextStyle.Bold = true
    return label
}

func (v *resultsView) rows() []reliability.Row {
    if v.current == nil {
        return nil
    }
    return v.current.Result.Rows
}

func (v *resultsView) saveCSV() {
    if v.current == nil {
        v.host.reportError(errNothingToSave)
        return
    }
    save := dialog.NewFileSave(func(target fyne.URIWriteCloser, err error) {
        if err != nil {
            v.host.reportError(err)
            return
        }
        if target == nil { // the dialog was cancelled
            return
        }
        v.host.reportError(writeResults(target, v.current.Result))
    }, v.host.window)
    save.SetFileName(resultsFileName)
    save.Show()
}

func writeResults(target io.WriteCloser, result *reliability.Result) error {
    return errors.Join(export.WriteCSV(target, exportRecords(result)), target.Close())
}
```

internal/ui/result_table.go

```
package ui

import (
    "strconv"

    "empirical-reliability/internal/numfmt"
    "empirical-reliability/internal/reliability"
)
```

```
9
10  // resultColumn describes one column of the results table. Showing another
11  // value only takes a new entry in resultColumns.
12  type resultColumn struct {
13      header string
14      width  float32
15      cell   func(index int, row reliability.Row) string
16  }
17
18  var resultColumns = []resultColumn{
19      {"i", 50, func(index int, _ reliability.Row) string { return strconv.Itoa(index + 1) }},
20      {"t_i", 90, func(_ int, row reliability.Row) string { return numfmt.Compact(row.Time) }},
21      {"Δt", 80, func(_ int, row reliability.Row) string { return numfmt.Compact(row.Step) }},
22      {"m(t_i)", 70, func(_ int, row reliability.Row) string { return strconv.Itoa(row.Failures) }},
23      {"n(t_i)", 70, func(_ int, row reliability.Row) string { return strconv.Itoa(row.Working) }},
24      {"F*(t_i)", 95, func(_ int, row reliability.Row) string { return numfmt.Fixed(row.CDF,
         probabilityDecimals) }},
25      {"R*(t_i)", 95, func(_ int, row reliability.Row) string { return numfmt.Fixed(row.Reliability,
         probabilityDecimals) }},
26      {"f*(t_i)", 105, func(_ int, row reliability.Row) string { return numfmt.Fixed(row.Density,
         rateDecimals) }},
27      {"λ*(t_i)", 105, func(_ int, row reliability.Row) string { return numfmt.Fixed(row.Hazard,
         rateDecimals) }},
28  }
29
30  func resultHeaders() []string {
31      headers := make([]string, len(resultColumns))
32      for i, column := range resultColumns {
33          headers[i] = column.header
34      }
35      return headers
36  }
37
38  func resultCells(index int, row reliability.Row) []string {
39      cells := make([]string, len(resultColumns))
40      for i, column := range resultColumns {
41          cells[i] = column.cell(index, row)
42      }
43      return cells
44  }
45
46  // exportRecords returns the results table followed by the summary values, in
47  // the form written to a CSV file.
48  func exportRecords(result *reliability.Result) [][]string {
49      records := [][]string{resultHeaders()}
50      for i, row := range result.Rows {
51          records = append(records, resultCells(i, row))
52      }
53      return append(records,
54          []string{},
55          []string{exportSampleSize, strconv.Itoa(result.SampleSize)},
56          []string{exportMean, numfmt.Fixed(result.Mean, exportMeanDecimals)},
57          []string{exportMeanFromReliability, numfmt.Fixed(result.MeanFromReliability,
         exportMeanDecimals)},
58      )
59  }
```

internal/ui/summary.go

```
1   package ui
2
3   import (
4       "fmt"
5       "strings"
6
7       "empirical-reliability/internal/analysis"
8       "empirical-reliability/internal/numfmt"
9       "empirical-reliability/internal/reliability"
10  )
11
12  const summaryLineSeparator = "\n"
13
```

```go
// summaryText describes an analysis in a few lines shown above the results table.
func summaryText(a *analysis.Analysis) string {
    lines := sampleLines(a.Result)
    lines = append(lines, meanLines(a.Result)...)
    if a.Random != nil {
        lines = append(lines, randomLines(a.Random)...)
    }
    return strings.Join(lines, summaryLineSeparator)
}

func sampleLines(result *reliability.Result) []string {
    observationEnd := result.Rows[len(result.Rows)-1].Time
    return []string{
        fmt.Sprintf(summarySampleFormat, result.SampleSize, len(result.Rows),
     numfmt.Compact(observationEnd)),
        fmt.Sprintf(summaryCountsFormat, result.Failed, result.Survived),
    }
}

func meanLines(result *reliability.Result) []string {
    mean := numfmt.Fixed(result.Mean, meanDecimals)
    fromReliability := numfmt.Fixed(result.MeanFromReliability, meanDecimals)
    switch {
    case result.Survived > 0:
        return []string{summaryMeanUnknown, fmt.Sprintf(summaryMeanLowerBoundFormat, fromReliability)}
    case len(result.Times) == 0:
        // Grouped data carry no individual lifetimes.
        return []string{
            fmt.Sprintf(summaryMeanFromInspectionsFormat, mean),
            fmt.Sprintf(summaryMeanFromReliabilityFormat, fromReliability),
        }
    default:
        return []string{
            fmt.Sprintf(summaryMeanFormat, mean),
            fmt.Sprintf(summaryMeanFromReliabilityFormat, fromReliability),
        }
    }
}

func randomLines(sample *analysis.RandomSample) []string {
    parameters := make([]string, len(sample.Parameters))
    for i, value := range sample.Parameters {
        parameters[i] = fmt.Sprintf(summaryParameterFormat, sample.Spec.Parameters[i].Symbol,
     numfmt.Compact(value))
    }
    return []string{
        fmt.Sprintf(summaryDistributionFormat,
            distributionNames[sample.Spec.ID],
            strings.Join(parameters, summaryParameterSeparator),
            sample.Seed),
        fmt.Sprintf(summaryTheoreticalMeanFormat, numfmt.Fixed(sample.Model.Mean(), meanDecimals)),
    }
}
```

internal/ui/charts_view.go

```go
package ui

import (
    "fmt"
    "image"
    "path/filepath"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/canvas"
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/dialog"
    "fyne.io/fyne/v2/widget"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/chart"
    "empirical-reliability/internal/export"
```

```
)

// chartsView shows one chart per reliability indicator.
type chartsView struct {
    host     host
    panels   map[chart.Indicator]*canvas.Image
    rendered map[chart.Indicator]image.Image // empty until something is computed
}

func newChartsView(h host) *chartsView {
    v := &chartsView{host: h, panels: map[chart.Indicator]*canvas.Image{}}
    for _, indicator := range chart.Indicators() {
        v.panels[indicator] = newChartPanel()
    }
    return v
}

func newChartPanel() *canvas.Image {
    panel := canvas.NewImageFromImage(nil)
    panel.FillMode = canvas.ImageFillContain
    panel.SetMinSize(fyne.NewSize(chartMinWidth, chartMinHeight))
    return panel
}

func (v *chartsView) tab() *container.TabItem {
    grid := container.NewGridWithColumns(chartColumns)
    for _, indicator := range chart.Indicators() {
        grid.Add(v.panels[indicator])
    }
    save := widget.NewButton(buttonSaveCharts, v.saveCharts)
    return container.NewTabItem(tabCharts, container.NewBorder(nil, save, nil, nil, grid))
}

func (v *chartsView) show(a *analysis.Analysis) {
    rendered, err := renderCharts(a)
    if err != nil {
        v.host.reportError(err)
        return
    }
    v.rendered = rendered
    for indicator, img := range rendered {
        v.panels[indicator].Image = img
        v.panels[indicator].Refresh()
    }
}

func renderCharts(a *analysis.Analysis) (map[chart.Indicator]image.Image, error) {
    rendered := map[chart.Indicator]image.Image{}
    for _, indicator := range chart.Indicators() {
        img, err := chart.Render(indicator, a.Result, a.Model(), chartLabels(indicator))
        if err != nil {
            return nil, err
        }
        rendered[indicator] = img
    }
    return rendered, nil
}

func (v *chartsView) saveCharts() {
    if len(v.rendered) == 0 {
        v.host.reportError(errNothingToSave)
        return
    }
    dialog.ShowFolderOpen(func(folder fyne.ListableURI, err error) {
        if err != nil {
            v.host.reportError(err)
            return
        }
        if folder == nil { // the dialog was cancelled
            return
        }
        if err := v.saveTo(folder.Path()); err != nil {
            v.host.reportError(err)
```

```
            return
        }
        v.host.inform(dialogSavedTitle, fmt.Sprintf(dialogChartsSavedFormat, folder.Path()))
    }, v.host.window)
}

func (v *chartsView) saveTo(folder string) error {
    for indicator, img := range v.rendered {
        path := filepath.Join(folder, indicator.Slug()+chartFileExtension)
        if err := export.SavePNG(path, img); err != nil {
            return err
        }
    }
    return nil
}
```

internal/ui/sample_view.go

```
package ui

import (
    "fmt"
    "strings"

    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/widget"

    "empirical-reliability/internal/analysis"
    "empirical-reliability/internal/numfmt"
)

const sampleSeparator = "\n"

// sampleView lists the individual failure times of the analysed sample.
type sampleView struct {
    times *widget.Entry
}

func newSampleView() *sampleView {
    return &sampleView{times: multiLineEntry(placeholderSample)}
}

func (v *sampleView) tab() *container.TabItem {
    hint := wrappedLabel(fmt.Sprintf(hintSampleFormat, tabTimes))
    return container.NewTabItem(tabSample, container.NewBorder(hint, nil, nil, nil, v.times))
}

func (v *sampleView) show(a *analysis.Analysis) {
    formatted := make([]string, len(a.Result.Times))
    for i, t := range a.Result.Times {
        formatted[i] = numfmt.Compact(t)
    }
    v.times.SetText(strings.Join(formatted, sampleSeparator))
}
```

Teksty i komunikaty

internal/ui/texts.go

```
package ui

import (
    "empirical-reliability/internal/chart"
    "empirical-reliability/internal/distribution"
)

// All texts shown to the user are collected in this file. The application is
// presented in Polish; the rest of the code base is in English.

const (
    windowTitle = "Empiryczne wskaźniki niezawodności"
```

```
13
14      tabRandom  = "Próba losowa"
15      tabTable   = "Tabela uszkodzeń"
16      tabTimes   = "Lista czasów"
17      tabResults = "Wyniki"
18      tabCharts  = "Wykresy"
19      tabSample  = "Próba"
20
21      labelDistribution    = "Rozkład trwałości"
22      labelRandomSize      = "Liczność próby n"
23      labelSampleSize      = "Liczność próby n(0)"
24      labelStep            = "Długość przedziału Δt"
25      labelSeed            = "Ziarno generatora"
26      parameterLabelFormat = "%s (%s)"
27
28      placeholderDistribution = "wybierz rozkład"
29      placeholderAutomatic    = "automatycznie"
30      placeholderRandomSeed   = "losowe"
31      placeholderTimesCount   = "liczba podanych czasów"
32      placeholderTimes        = "np. 12,5 30 41,2 55 ..."
33      placeholderSample       = "Tu pojawią się pojedyncze czasy uszkodzeń (posortowane rosnąco)."
34
35      buttonDraw       = "Losuj i oblicz"
36      buttonCompute    = "Oblicz"
37      buttonExample    = "Przykład z książki"
38      buttonClear      = "Wyczyść"
39      buttonSaveCSV    = "Zapisz wyniki do pliku CSV"
40      buttonSaveCharts = "Zapisz wykresy jako pliki PNG"
41
42      hintRandom = "Program losuje n czasów do uszkodzenia z wybranego rozkładu, " +
43          "grupuje je w przedziały o długości Δt i wyznacza wskaźniki empiryczne. " +
44          "To samo ziarno daje zawsze tę samą próbę."
45      hintTable = "W każdym wierszu: chwila t oraz liczba elementów uszkodzonych m(t) " +
46          "zarejestrowana w tej chwili, np. „2 3"."
47      hintTimes = "Czasy do uszkodzenia kolejnych elementów, rozdzielone spacjami, " +
48          "średnikami lub znakami nowej linii. Jeżeli n(0) jest większe od liczby czasów, " +
49          "pozostałe elementy są traktowane jako zdatne do końca obserwacji."
50      hintSampleFormat = "Czasy uszkodzeń z próby, posortowane rosnąco. " +
51          "Można je skopiować do zakładki „%s"."
52
53      summaryPrompt                    = "Wprowadź dane po lewej stronie i wybierz „" + buttonCompute +
        ""."
54      summarySampleFormat              = "Liczność próby n(0) = %d, liczba przedziałów: %d, czas
        obserwacji: %s"
55      summaryCountsFormat              = "Elementy uszkodzone: %d, zdatne po zakończeniu obserwacji: %d"
56      summaryMeanFormat                = "E*T ze wzoru (5-15) = %s"
57      summaryMeanFromInspectionsFormat = "E*T ze wzoru (5-15) = %s (za trwałość elementu przyjęto
        chwilę rejestracji uszkodzenia)"
58      summaryMeanUnknown               = "E*T ze wzoru (5-15): nie można wyznaczyć, ponieważ nie
        wszystkie elementy uległy uszkodzeniu"
59      summaryMeanFromReliabilityFormat = "E*T z funkcji R*(t) = %s"
60      summaryMeanLowerBoundFormat      = "E*T z funkcji R*(t) = %s (oszacowanie od dołu)"
61      summaryDistributionFormat        = "Rozkład %s, %s; ziarno generatora: %d"
62      summaryParameterFormat           = "%s = %s"
63      summaryParameterSeparator        = ", "
64      summaryTheoreticalMeanFormat     = "Teoretyczna wartość oczekiwana ET = %s"
65
66      exportSampleSize          = "n(0)"
67      exportMean                = "E*T (5-15)"
68      exportMeanFromReliability = "E*T z R*(t)"
69
70      dialogSavedTitle        = "Zapisano"
71      dialogChartsSavedFormat = "Wykresy zapisano w folderze:\n%s"
72
73      chartTimeAxis    = "t"
74      chartEmpirical   = "empiryczna"
75      chartTheoretical = "teoretyczna"
76  )
77
78  var (
79      distributionNames = map[string]string{
80          distribution.Exponential.ID: "wykładniczy",
81          distribution.Weibull.ID:     "Weibulla",
```

```
        distribution.Normal.ID:      "normalny",
    }

    // parameterNames describes distribution parameters by their symbol.
    parameterNames = map[string]string{
        "λ": "intensywność uszkodzeń",
        "β": "kształt",
        "η": "skala",
        "μ": "wartość średnia",
        "σ": "odchylenie standardowe",
    }

    chartTitles = map[chart.Indicator]string{
        chart.CDF:         "Dystrybuanta empiryczna F*(t)",
        chart.Reliability: "Funkcja niezawodności R*(t)",
        chart.Density:     "Gęstość prawdopodobieństwa f*(t)",
        chart.Hazard:      "Intensywność uszkodzeń λ*(t)",
    }
)

func chartLabels(indicator chart.Indicator) chart.Labels {
    return chart.Labels{
        Title:       chartTitles[indicator],
        TimeAxis:    chartTimeAxis,
        Empirical:   chartEmpirical,
        Theoretical: chartTheoretical,
    }
}
```

internal/ui/errors.go

```
package ui

import (
    "errors"
    "fmt"

    "empirical-reliability/internal/distribution"
    "empirical-reliability/internal/input"
    "empirical-reliability/internal/numfmt"
    "empirical-reliability/internal/reliability"
)

const (
    errorFieldFormat     = "%s: %s"
    errorLineFormat      = "wiersz %d: %s"
    errorItemFormat      = "pozycja %d: %s"
    errorSyntaxFormat    = "„%s" %s"
    errorParameterFormat = "parametr %s musi być dodatni"
)

// errNothingToSave reports an attempt to save before anything was computed.
var errNothingToSave = errors.New("nothing to save")

// errorMessages translates the errors of the lower layers, which are written
// for developers, into messages for the user.
var errorMessages = map[error]string{
    reliability.ErrSampleSizeNotPositive:  "liczność próby n(0) musi być liczbą dodatnią",
    reliability.ErrNoObservations:         "tabela uszkodzeń jest pusta",
    reliability.ErrTimeNotIncreasing:      "chwile t muszą być dodatnie i rosnące",
    reliability.ErrNegativeFailures:       "liczba uszkodzeń nie może być ujemna",
    reliability.ErrFailuresExceedSample:   "łączna liczba uszkodzeń przekracza liczność próby n(0)",
    reliability.ErrNoFailureTimes:         "nie podano żadnych czasów uszkodzeń",
    reliability.ErrFailureTimeNotPositive: "czas uszkodzenia musi być dodatni",
    reliability.ErrStepNotPositive:        "długość przedziału Δt musi być dodatnia",
    reliability.ErrTooManyIntervals: fmt.Sprintf(
        "zbyt mała długość przedziału Δt (ponad %d przedziałów)", reliability.MaxIntervals),
    reliability.ErrSampleSmallerThanTimes: "liczność próby n(0) jest mniejsza od liczby podanych
     czasów",

    distribution.ErrSampleSizeNotPositive: "liczność próby musi być liczbą dodatnią",
    distribution.ErrSampleSizeTooLarge: fmt.Sprintf(
```

```
        "liczność próby nie może przekraczać %d", distribution.MaxSampleSize),
    distribution.ErrParameterCount: "niepoprawna liczba parametrów rozkładu",

    input.ErrColumnCount: "oczekiwano dwóch wartości: t oraz m(t)",

    numfmt.ErrNotNumber:   "nie jest poprawną liczbą",
    numfmt.ErrNotInteger:  "nie jest poprawną liczbą całkowitą",
    numfmt.ErrNotUnsigned: "nie jest nieujemną liczbą całkowitą",

    errNothingToSave: "najpierw wykonaj obliczenia",
}

// fieldError attaches the label of the form field an invalid value came from.
type fieldError struct {
    label string
    err   error
}

func (e *fieldError) Error() string { return fmt.Sprintf(errorFieldFormat, e.label, e.err) }
func (e *fieldError) Unwrap() error { return e.err }

// describe returns the message shown to the user for err. Errors that carry a
// location are unwrapped layer by layer, so the type switch deliberately
// inspects only the outermost error.
func describe(err error) string {
    switch e := err.(type) {
    case *fieldError:
        return fmt.Sprintf(errorFieldFormat, e.label, describe(e.err))
    case *input.LineError:
        return fmt.Sprintf(errorLineFormat, e.Line, describe(e.Err))
    case *reliability.ItemError:
        return fmt.Sprintf(errorItemFormat, e.Position, describe(e.Err))
    case *numfmt.SyntaxError:
        return fmt.Sprintf(errorSyntaxFormat, e.Text, describe(e.Err))
    case *distribution.ParameterError:
        return fmt.Sprintf(errorParameterFormat, e.Symbol)
    }
    for sentinel, message := range errorMessages {
        if errors.Is(err, sentinel) {
            return message
        }
    }
    return err.Error()
}
```

4 Opis i wyniki testowania aplikacji

Program sprawdzono na cztery sposoby: testami automatycznymi, porównaniem z przykładami z książki, porównaniem wskaźników empirycznych z teoretycznymi dla prób losowych oraz podaniem błędnych danych.

4.1 Testy automatyczne

Do programu dołączono 64 testy automatyczne, czyli krótkie programy sprawdzające, czy poszczególne części programu dają oczekiwane wyniki (tab. 2). Uruchamia się je jednym poleceniem i wykonują się w kilka sekund. Wszystkie testy kończą się powodzeniem.

Tabela 2: Zakres testów automatycznych

Część programu	Liczba	Co jest sprawdzane
wyznaczanie wskaźników	16	zgodność z przykładami 5.1–5.4, przedziały o różnej długości, próba niepełna, odrzucanie błędnych danych, grupowanie czasów (w tym czas na granicy przedziału), automatyczny dobór Δt
rozkłady trwałości	8	odrzucanie niedodatnich parametrów, zbieżność średniej z próby do ET, monotoniczność dystrybuant, stała intensywność rozkładu wykładniczego
losowanie próby	6	powtarzalność próby dla tego samego ziarna, zbieżność F^* do F oraz E^*T do ET dla $n = 50\,000$
wczytywanie danych	6	separatory, puste wiersze, błędne wiersze i numer wiersza w komunikacie
zapis liczb	7	przecinek i kropka dziesiętna, błędne liczby, zapis wyników
wykresy	6	rysowanie wszystkich wykresów, pominięcie nieokreślonej λ^*, podziałka osi
zapis plików	3	format pliku CSV, zapis pliku PNG
okno programu	12	obliczenia uruchamiane przyciskiem z każdego formularza, treść komunikatów o błędach, zapis wyników i wykresów

Testy okna programu uruchamiają je w trybie testowym, wypełniają formularze i „naciskają” przyciski, a następnie sprawdzają wyświetlone wyniki. Zrzuty okna zamieszczone w tym sprawozdaniu zostały zapisane podczas tych testów.

4.2 Zgodność z przykładami z książki

Dane z przykładu 5.1 w [1]: próbka $n(0) = 35$ kondensatorów elektrolitycznych, badanie do uszkodzenia ostatniego elementu, rejestracja co godzinę; liczby uszkodzeń w kolejnych godzinach: 0, 3, 3, 5, 8, 7, 6, 2, 1, 0. Dane te są wczytywane w zakładce „Tabela uszkodzeń” po uruchomieniu programu (rys. 2 i 3). W tab. 3 i 4 zestawiono wyniki programu z tablicami z przykładów 5.1–5.4.

Tabela 3: Dystrybuanta i funkcja niezawodności: program i książka (przykłady 5.1 i 5.2)

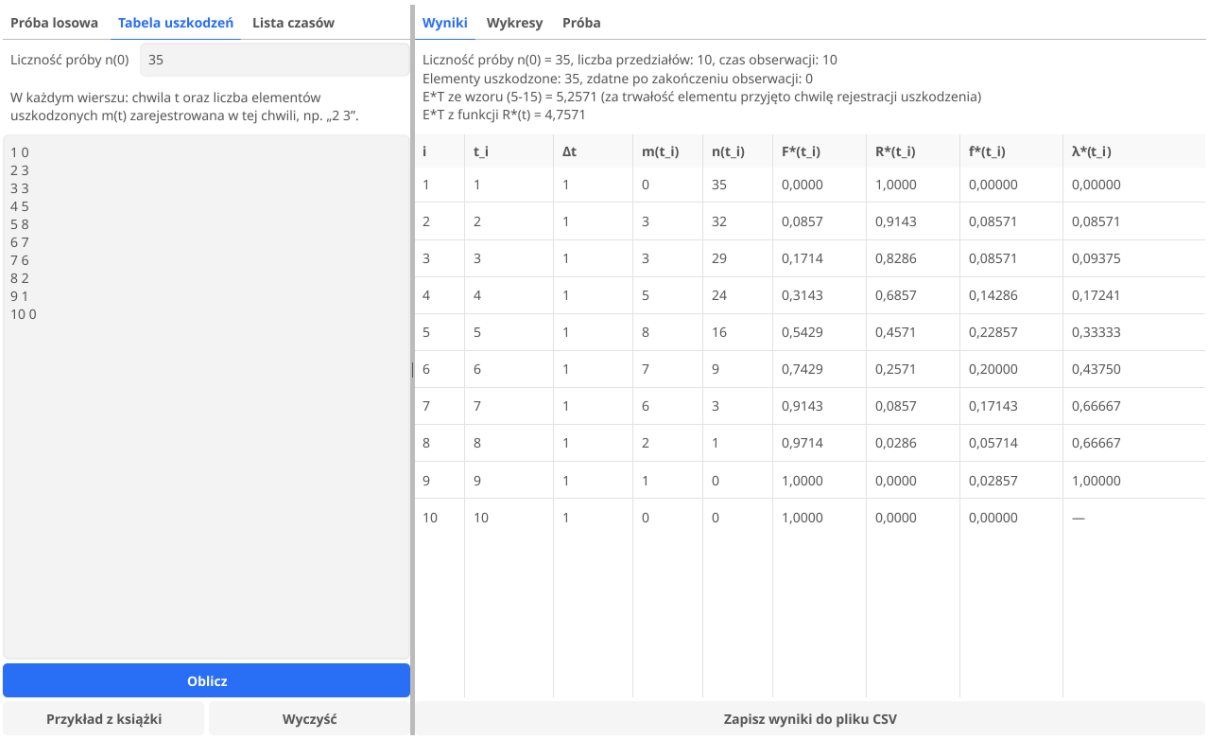
		$F^*(t_i)$		$R^*(t_i)$	
t_i [h]	$m(t_i)$	program	książka	program	książka
1	0	0,0000	0	1,0000	1
2	3	0,0857	0,086	0,9143	0,914
3	3	0,1714	0,171	0,8286	0,828
4	5	0,3143	0,314	0,6857	0,686
5	8	0,5429	0,543	0,4571	0,457
6	7	0,7429	0,743	0,2571	0,257
7	6	0,9143	0,914	0,0857	0,086
8	2	0,9714	0,971	0,0286	0,029
9	1	1,0000	1	0,0000	0
10	0	1,0000	1	0,0000	0

Tabela 4: Gęstość i intensywność uszkodzeń: program i książka (przykłady 5.3 i 5.4)

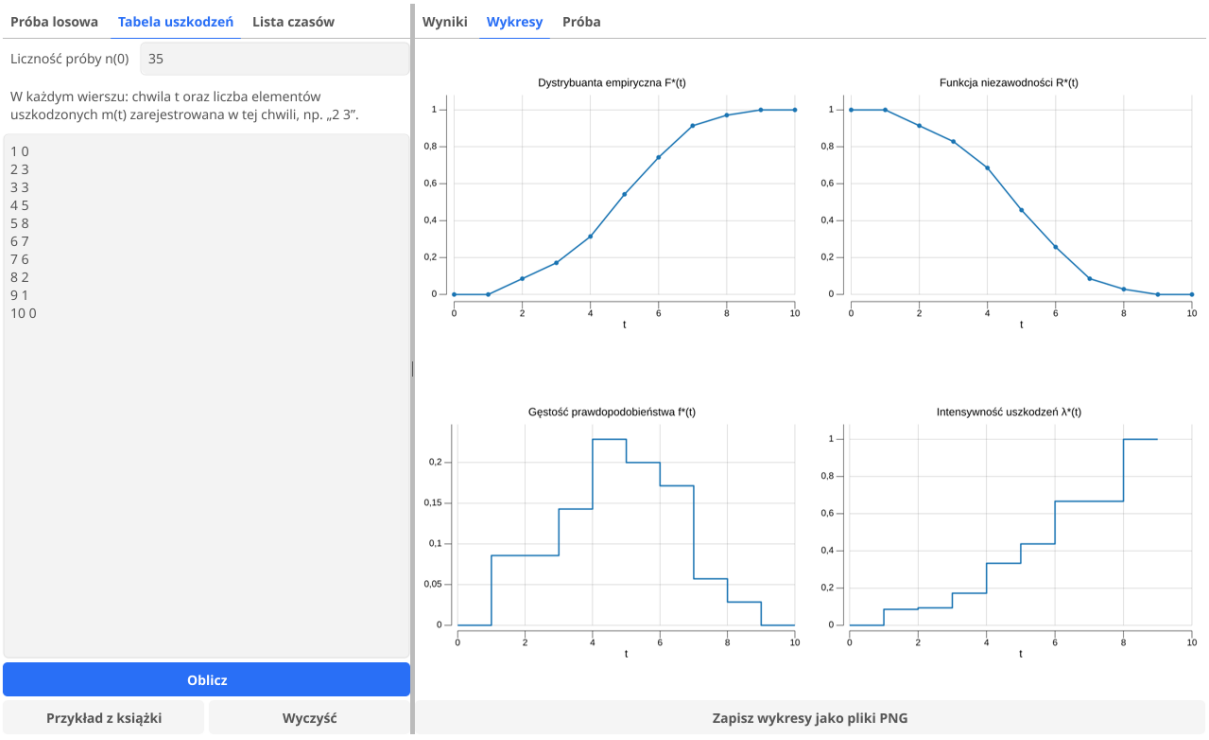
	$f^*(t_i)$ [1/h]		$\lambda^*(t_i)$ [1/h]	
t_i [h]	program	książka	program	książka
1	0,00000	0	0,00000	0
2	0,08571	0,086	0,08571	0,086
3	0,08571	0,086	0,09375	0,095
4	0,14286	0,143	0,17241	0,172
5	0,22857	0,228	0,33333	0,333
6	0,20000	0,2	0,43750	0,437
7	0,17143	0,171	0,66667	0,667
8	0,05714	0,057	0,66667	0,667
9	0,02857	0,0286	1,00000	1,0
10	0,00000	—	—	—

Wyniki programu zgadzają się z książką. Największa różnica wynosi 0,00125 i dotyczy $\lambda^*(t_3) = 3/32 = 0{,}09375$, dla której w książce podano 0,095. Pozostałe różnice są na trzecim miejscu po przecinku i wynikają z obcinania cyfr w książce, np. $R^*(t_3) = 29/35 = 0{,}82857$ (w książce 0,828) oraz $f^*(t_5) = 8/35 = 0{,}22857$ (w książce 0,228). Dla t_{10} książka nie podaje f^* ani λ^*; program wyznacza $f^*(t_{10}) = 0$, a $\lambda^*(t_{10})$ oznacza jako nieokreśloną, ponieważ po dziewiątej godzinie nie ma już elementów zdatnych.

Dla tych danych program podaje $E^*T = 5{,}2571$ h ze wzoru (5-15), przyjmując za trwałość chwilę rejestracji uszkodzenia (184/35), oraz $E^*T = 4{,}7571$ h z całkowania $R^*(t)$. Różnica wynosi dokładnie 0,5 h, czyli połowę odstępu między rejestracjami: pierwsza wartość zakłada, że elementy uszkadzały się na końcu każdej godziny, a druga – że równomiernie w jej trakcie. Poprawność obu wartości sprawdzono rachunkiem ręcznym.



Rysunek 2: Wyniki programu dla danych z przykładu 5.1



Rysunek 3: Wykresy wskaźników dla danych z przykładu 5.1

4.3 Próba losowa a rozkład teoretyczny

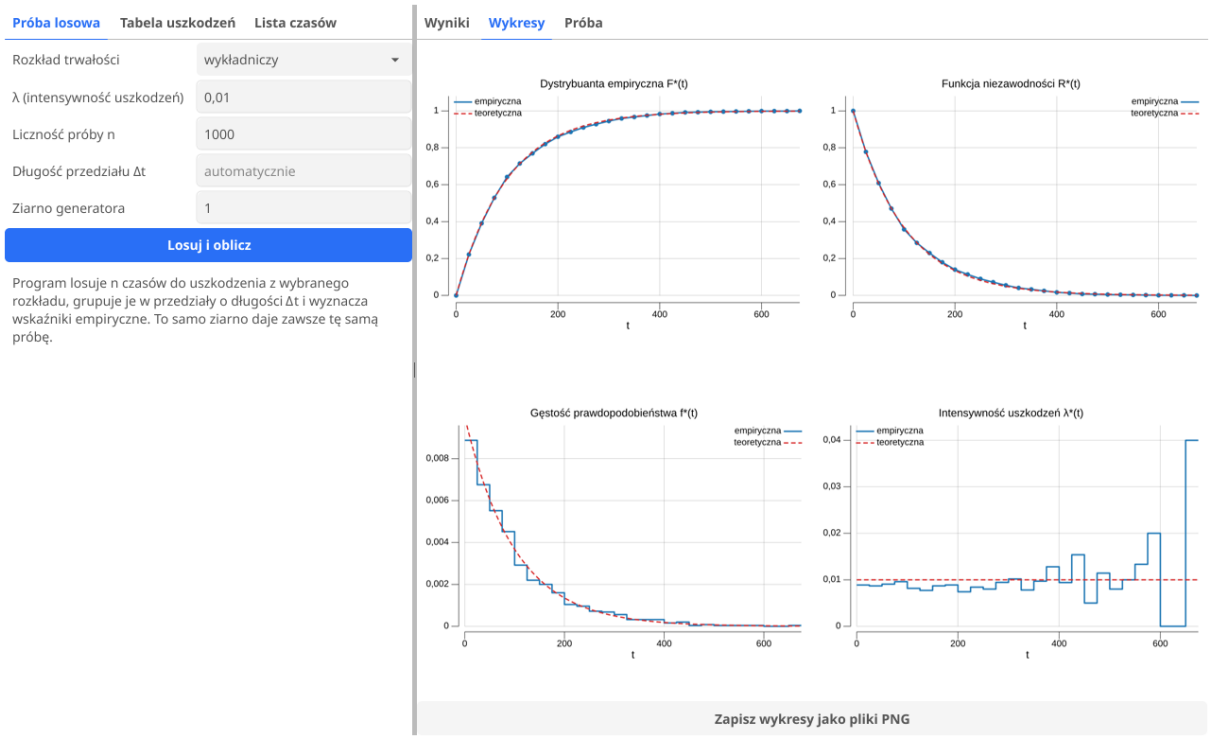
Dla każdego z trzech rozkładów wylosowano próby o liczności 100, 1000 i 10 000 (ziarno generatora 1, parametry domyślne, Δt dobierane automatycznie) i porównano wskaźniki

empiryczne z teoretycznymi (tab. 5). Wyniki można odtworzyć w programie, wpisując te same ustawienia.

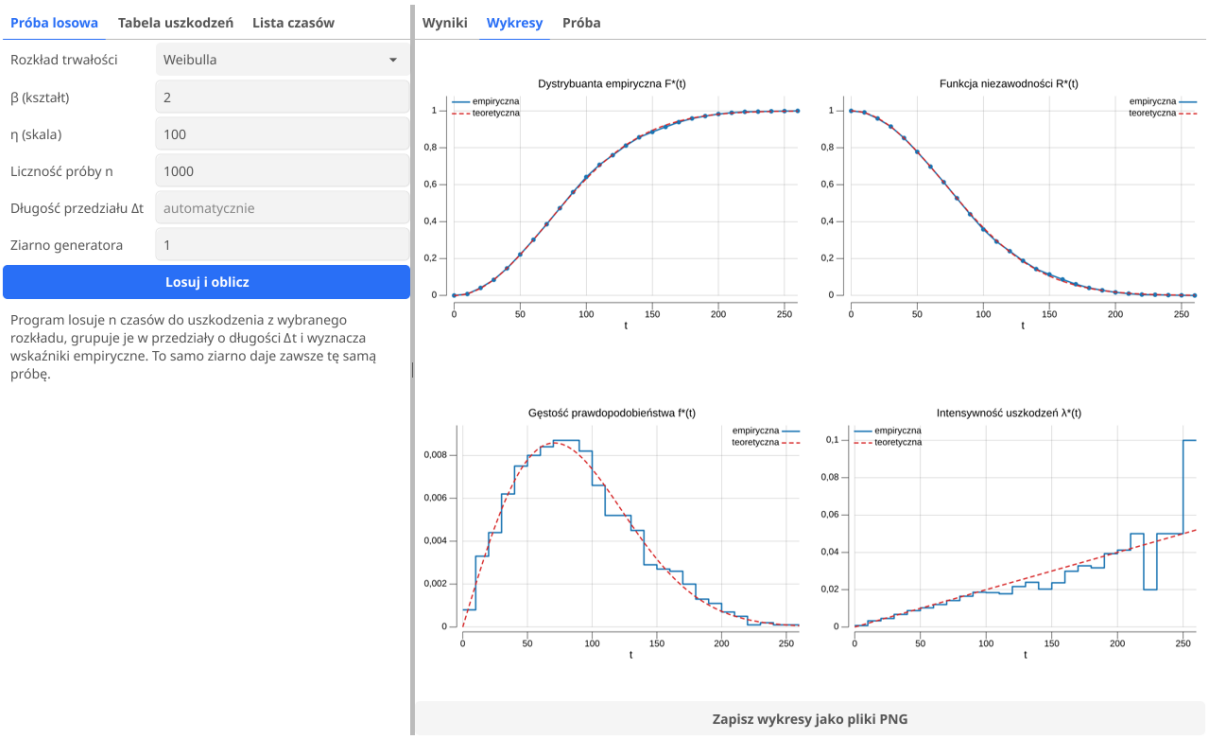
Tabela 5: Wskaźniki empiryczne dla prób losowych; $\max|F^* - F|$ to największa różnica między dystrybuantą empiryczną i teoretyczną w chwilach t_i

Rozkład	n	Δt	k	E^*T (5-15)	E^*T z R^*	ET	$\max\|F^* - F\|$
wykładniczy	100	50	10	104,82	103,50	100,00	0,0479
$\lambda = 0{,}01$	1000	25	27	100,57	101,08	100,00	0,0099
	10 000	20	47	99,00	99,30	100,00	0,0043
Weibulla	100	25	9	90,94	89,00	88,62	0,0479
$\beta = 2$, $\eta = 100$	1000	10	26	88,93	88,67	88,62	0,0099
	10 000	10	31	88,26	88,22	88,62	0,0049
normalny	100	20	7	101,30	100,40	100,00	0,0513
$\mu = 100$, $\sigma = 20$	1000	10	17	100,23	100,22	100,00	0,0125
	10 000	5	37	99,71	99,73	100,00	0,0074

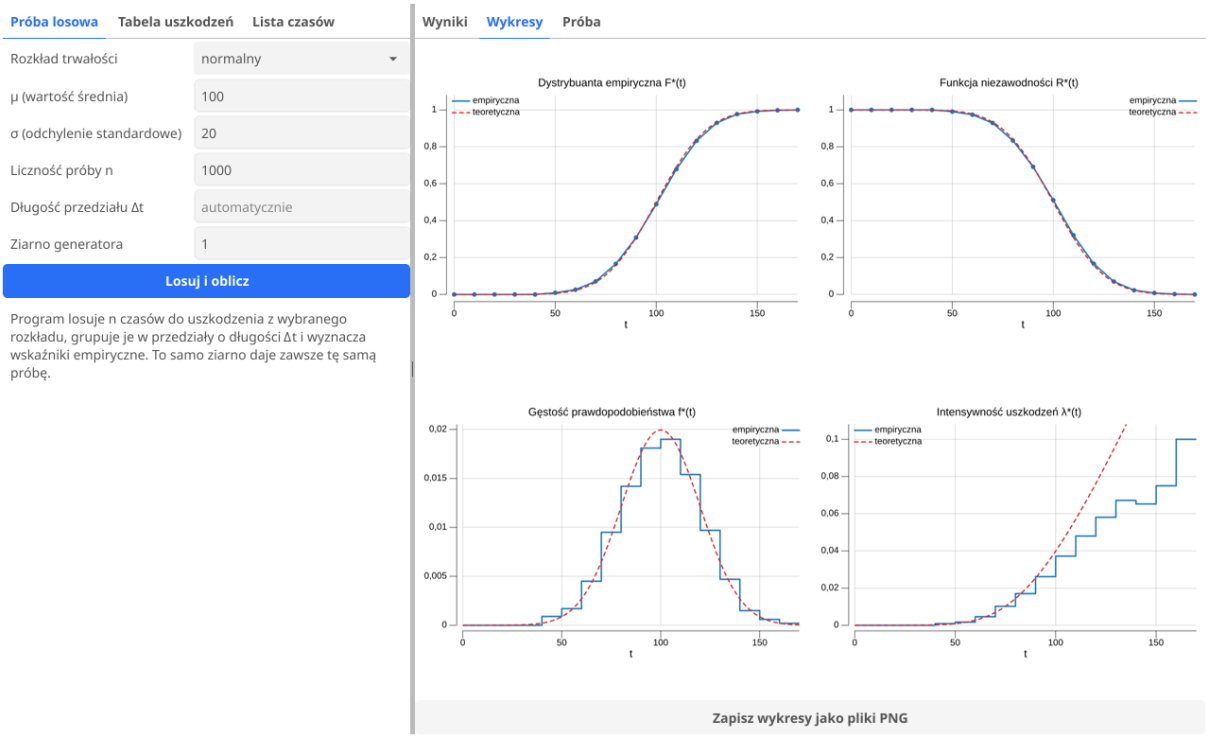
Wraz ze wzrostem liczności próby wskaźniki empiryczne zbliżają się do teoretycznych: największa różnica dystrybuant maleje z około 0,05 dla $n = 100$ do poniżej 0,01 dla $n = 10\,000$, a błąd względny E^*T – z kilku procent do najwyżej jednego procenta. Wykresy (rys. 4–6) pokazują zgodność przebiegów empirycznych (linia ciągła) z teoretycznymi (linia przerywana). Kształt $\lambda^*(t)$ odpowiada teorii: intensywność jest stała dla rozkładu wykładniczego i rośnie dla rozkładów Weibulla ($\beta = 2$) i normalnego. Na końcu osi czasu przebieg λ^* staje się nieregularny, ponieważ zostaje już tylko kilka zdatnych elementów i pojedyncze uszkodzenie silnie zmienia wynik.



Rysunek 4: Próba losowa z rozkładu wykładniczego, $n = 1000$, ziarno 1



Rysunek 5: Próba losowa z rozkładu Weibulla, $n = 1000$, ziarno 1



Rysunek 6: Próba losowa z rozkładu normalnego, $n = 1000$, ziarno 1

4.4 Wpływ długości przedziału na wyniki

Na rys. 4 empiryczna intensywność uszkodzeń leży nieco poniżej teoretycznej. Nie jest to błąd programu, lecz cecha wzoru (5-12): odnosi on uszkodzenia z całego przedziału

do liczby elementów zdatnych na jego początku, podczas gdy w trakcie przedziału elementów zdatnych ubywa. Dla rozkładu wykładniczego udział elementów uszkodzonych w przedziale wynosi średnio $1 - e^{-\lambda\Delta t}$, więc wzór (5-12) daje przeciętnie

$$\lambda^* \approx \frac{1 - e^{-\lambda\Delta t}}{\Delta t} < \lambda.$$

Sprawdzono to dla próby z rozkładu wykładniczego ($\lambda = 0{,}01$, $n = 10\,000$, ziarno 1), zmieniając tylko długość przedziału (tab. 6).

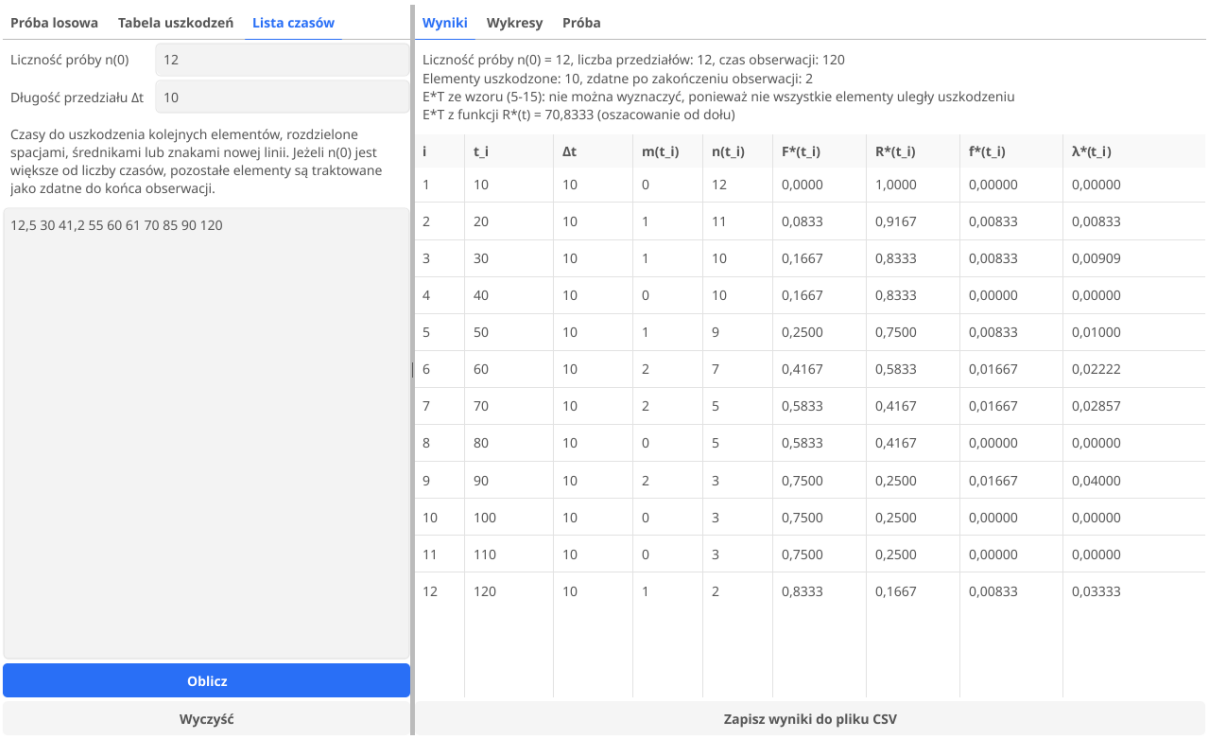
Tabela 6: Wpływ długości przedziału Δt na wyniki; rozkład wykładniczy, $\lambda = 0{,}01$, $ET = 100$

Δt	$\lambda^*(t_1)$	$\lambda^*(t_2)$	$\lambda^*(t_3)$	$\frac{1 - e^{-\lambda\Delta t}}{\Delta t}$	E^*T z R^*
100	0,00635	0,00636	0,00640	0,00632	107,15
50	0,00795	0,00787	0,00788	0,00787	101,01
25	0,00885	0,00906	0,00880	0,00885	99,50
10	0,00942	0,00966	0,00951	0,00952	99,09
5	0,00984	0,00947	0,00991	0,00975	99,00

Wartości wyznaczone przez program zgadzają się z przewidywanymi. Dla $\Delta t = 100$ intensywność jest zaniżona o blisko 37%, a dla $\Delta t = 5$ – o około 2,5%. Podobnie zachowuje się E^*T wyznaczana z $R^*(t)$: przy grubych przedziałach metoda trapezów zawyża pole pod wypukłą krzywą, a wraz ze zmniejszaniem Δt wynik zbliża się do wartości 98,9976 otrzymanej ze wzoru (5-15), która od Δt nie zależy. Przedział nie może być jednak dowolnie krótki: gdy w przedziale jest niewiele uszkodzeń, wartości f^* i λ^* stają się przypadkowe. Dlatego program domyślnie dobiera około $\sqrt{n}$ przedziałów.

4.5 Próba niepełna

Sprawdzono przypadek, w którym badanie zakończono przed uszkodzeniem wszystkich elementów. W zakładce „Lista czasów” podano $n(0) = 12$, $\Delta t = 10$ oraz dziesięć czasów uszkodzeń: 12,5; 30; 41,2; 55; 60; 61; 70; 85; 90; 120 (rys. 7).



i	t_i	Δt	m(t_i)	n(t_i)	F*(t_i)	R*(t_i)	f*(t_i)	λ*(t_i)
1	10	10	0	12	0,0000	1,0000	0,00000	0,00000
2	20	10	1	11	0,0833	0,9167	0,00833	0,00833
3	30	10	1	10	0,1667	0,8333	0,00833	0,00909
4	40	10	0	10	0,1667	0,8333	0,00000	0,00000
5	50	10	1	9	0,2500	0,7500	0,00833	0,01000
6	60	10	2	7	0,4167	0,5833	0,01667	0,02222
7	70	10	2	5	0,5833	0,4167	0,01667	0,02857
8	80	10	0	5	0,5833	0,4167	0,00000	0,00000
9	90	10	2	3	0,7500	0,2500	0,01667	0,04000
10	100	10	0	3	0,7500	0,2500	0,00000	0,00000
11	110	10	0	3	0,7500	0,2500	0,00000	0,00000
12	120	10	1	2	0,8333	0,1667	0,00833	0,03333

Rysunek 7: Wyniki dla listy czasów przy niepełnej próbie

Wyniki sprawdzono rachunkiem ręcznym. Przykładowo do przedziału $(50, 60]$ należą czasy 55 i 60 (czas leżący na granicy trafia do niższego przedziału), a na jego początku zdatnych jest 9 elementów, zatem

$$F^*(60) = \frac{5}{12} = 0{,}4167, \qquad f^*(60) = \frac{2}{12 \cdot 10} = 0{,}01667, \qquad \lambda^*(60) = \frac{2}{9 \cdot 10} = 0{,}02222,$$

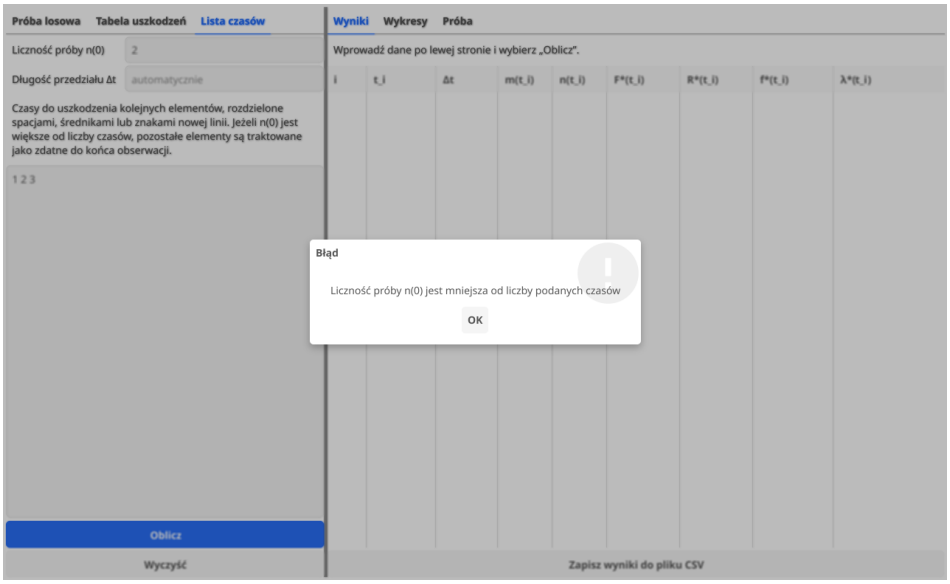
co zgadza się z szóstym wierszem tabeli. Dwa elementy pozostały zdatne, więc $R^*(120) = 2/12 = 0{,}1667$. Program poprawnie informuje, że E^*T ze wzoru (5-15) nie da się wyznaczyć, i podaje oszacowanie od dołu 70,83 z całkowania $R^*(t)$.

4.6 Obsługa błędnych danych

Program nie wykonuje obliczeń dla błędnych danych, tylko wyświetla okno z opisem błędu (rys. 8), a poprzednie wyniki pozostają bez zmian. Sprawdzone przypadki zestawiono w tab. 7.

Tabela 7: Reakcja programu na błędne dane

Wprowadzone dane	Komunikat
pusta tabela uszkodzeń	tabela uszkodzeń jest pusta
$n(0)$ = „abc”	Liczność próby n(0): „abc” nie jest poprawną liczbą całkowitą
tabela z przykładu 5.1 przy $n(0) = 10$	pozycja 4: łączna liczba uszkodzeń przekracza liczność próby n(0)
wiersz tabeli z jedną wartością	wiersz 2: oczekiwano dwóch wartości: t oraz m(t)
parametr $\lambda = -1$	parametr λ musi być dodatni
ziarno generatora „x”	Ziarno generatora: „x” nie jest nieujemną liczbą całkowitą
liczność próby losowej $n = 0$	liczność próby musi być liczbą dodatnią
$\Delta t = 0$	długość przedziału Δt musi być dodatnia
lista czasów „1 x 3”	pozycja 2: „x” nie jest poprawną liczbą
trzy czasy przy $n(0) = 2$	liczność próby n(0) jest mniejsza od liczby podanych czasów



Rysunek 8: Komunikat o błędnych danych

4.7 Wnioski

- Program odtwarza wyniki przykładów 5.1–5.4 z książki z dokładnością do zaokrągleń w niej zastosowanych.
- Dla prób losowych wskaźniki empiryczne zbliżają się do teoretycznych wraz ze wzrostem liczności próby, zgodnie z oczekiwaniami.
- Empiryczna intensywność uszkodzeń wyznaczana ze wzoru (5-12) jest zaniżona tym bardziej, im dłuższy jest przedział Δt; długość przedziału trzeba więc dobierać jako kompromis między tym błędem a rozrzutem wyników.
- Błędne dane są wykrywane przed obliczeniami i opisywane komunikatem wskazującym miejsce błędu.

5 Instrukcja obsługi programu

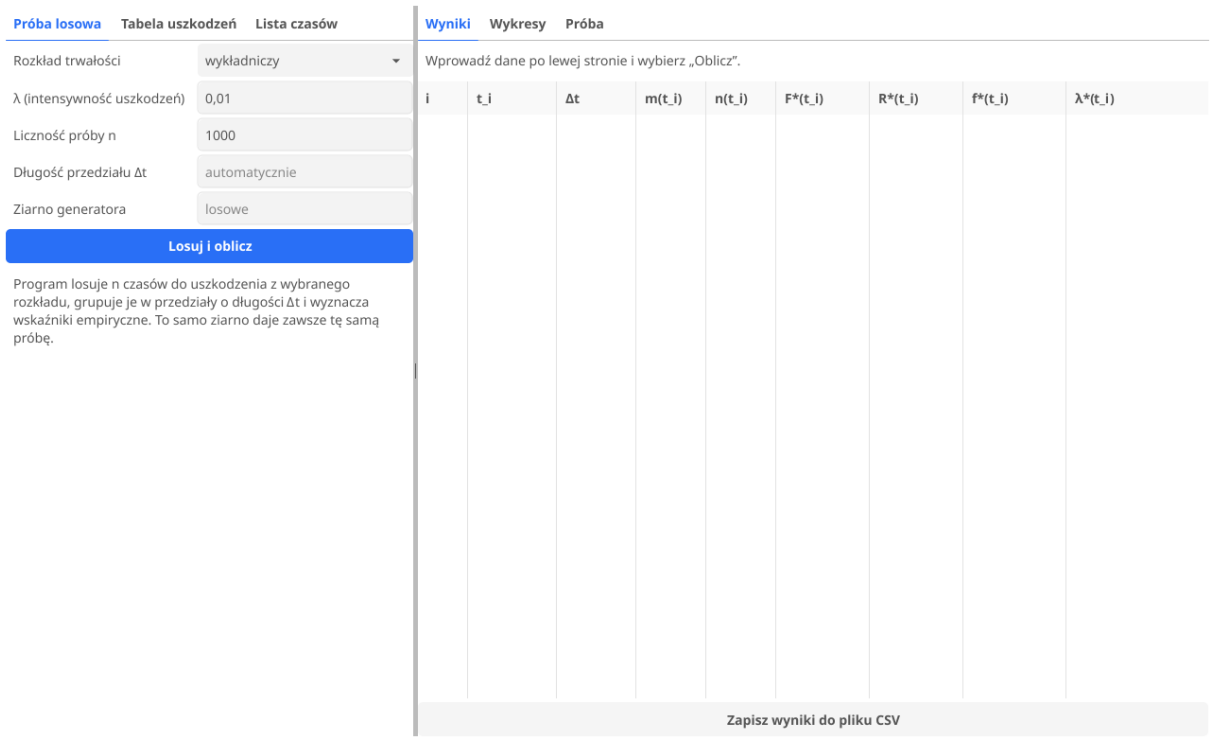
5.1 Uruchomienie

Program jest jednym plikiem wykonywalnym `empirical-reliability.exe` (Windows 10/11, 64 bity). Nie wymaga instalacji ani dodatkowych plików – wystarczy go uruchomić. Do działania potrzebny jest sterownik karty graficznej z obsługą OpenGL, dostępny standardowo w systemie. Wersję dla systemu Linux lub macOS buduje się z kodu źródłowego (p. 5.11).

5.2 Okno programu

Okno jest podzielone na dwie części (rys. 9). Po lewej stronie wprowadza się dane w jednej z trzech zakładek: „Próba losowa”, „Tabela uszkodzeń” albo „Lista czasów”. Po prawej stronie znajdują się wyniki w zakładkach „Wyniki”, „Wykresy” i „Próba”. Granicę między częściami można przesuwać myszą.

Liczby można wpisywać z przecinkiem albo z kropką dziesiętną (np. `0,01` lub `0.01`). Każde obliczenie zastępuje poprzednie wyniki.



Rysunek 9: Okno programu po uruchomieniu

5.3 Próba losowa

Zakładka służy do wylosowania czasów do uszkodzenia z rozkładu teoretycznego.

1. Wybrać **rozkład trwałości**: wykładniczy, Weibulla albo normalny.
2. Podać **parametry rozkładu** – pola zmieniają się zależnie od rozkładu i są wypełnione wartościami domyślnymi: λ dla wykładniczego; β (kształt) i η (skala) dla Weibulla; μ i σ dla normalnego. Wszystkie parametry muszą być dodatnie.

3. Podać **liczność próby** n (od 1 do 1 000 000).
4. Opcjonalnie podać **długość przedziału** Δt. Puste pole oznacza dobór automatyczny.
5. Opcjonalnie podać **ziarno generatora** (nieujemna liczba całkowita). To samo ziarno daje zawsze tę samą próbę; przy pustym polu każde losowanie jest inne, a użyte ziarno jest pokazywane w podsumowaniu wyników.
6. Nacisnąć **„Losuj i oblicz"**.

5.4 Tabela uszkodzeń

Zakładka służy do wprowadzenia danych pogrupowanych, takich jak w przykładach z książki.

1. Podać **liczność próby** $n(0)$.
2. W polu tekstowym wpisać tabelę: w każdym wierszu chwilę t oraz liczbę elementów uszkodzonych $m(t)$ zarejestrowaną w tej chwili, rozdzielone spacją, tabulatorem lub średnikiem, np. `2 3`. Chwile muszą być dodatnie i rosnące, ale odstępy między nimi nie muszą być równe. Puste wiersze są pomijane.
3. Nacisnąć **„Oblicz"**.

Suma uszkodzeń może być mniejsza od $n(0)$ – pozostałe elementy są wtedy traktowane jako zdatne po zakończeniu obserwacji. Przycisk „Przykład z książki" wczytuje dane z przykładu 5.1, a „Wyczyść" usuwa zawartość formularza.

5.5 Lista czasów

Zakładka służy do wprowadzenia pojedynczych czasów uszkodzeń.

1. W polu tekstowym wpisać czasy do uszkodzenia kolejnych elementów, rozdzielone spacjami, średnikami lub znakami nowej linii, w dowolnej kolejności.
2. Opcjonalnie podać **liczność próby** $n(0)$. Puste pole oznacza, że liczność jest równa liczbie podanych czasów. Jeżeli $n(0)$ jest większe, pozostałe elementy są traktowane jako zdatne do końca obserwacji.
3. Opcjonalnie podać **długość przedziału** Δt.
4. Nacisnąć **„Oblicz"**.

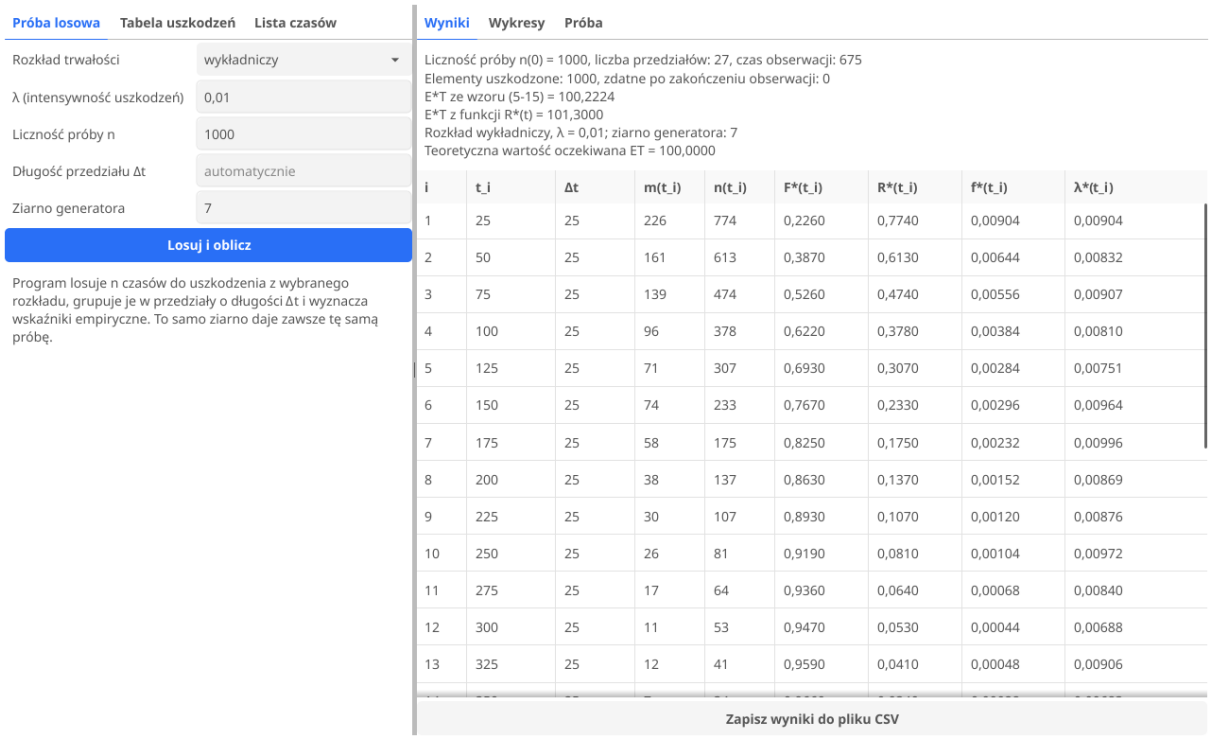
5.6 Wyniki

Zakładka „Wyniki" (rys. 10) zawiera podsumowanie i tabelę. W podsumowaniu podane są: liczność próby, liczba przedziałów i czas obserwacji, liczba elementów uszkodzonych i zdatnych, E^*T ze wzoru (5-15) oraz E^*T z funkcji $R^*(t)$, a dla próby losowej także rozkład, jego parametry, użyte ziarno i teoretyczna wartość oczekiwana ET. Kolumny tabeli opisano w tab. 8.

Tabela 8: Kolumny tabeli wyników

Kolumna	Znaczenie
i	numer przedziału
t_i	koniec przedziału (chwila rejestracji)
Δt	długość przedziału
$m(t_i)$	liczba elementów uszkodzonych w przedziale
$n(t_i)$	liczba elementów zdatnych w chwili t_i
$F^*(t_i)$	dystrybuanta empiryczna
$R^*(t_i)$	empiryczna funkcja niezawodności
$f^*(t_i)$	empiryczna gęstość prawdopodobieństwa
$\lambda^*(t_i)$	empiryczna intensywność uszkodzeń

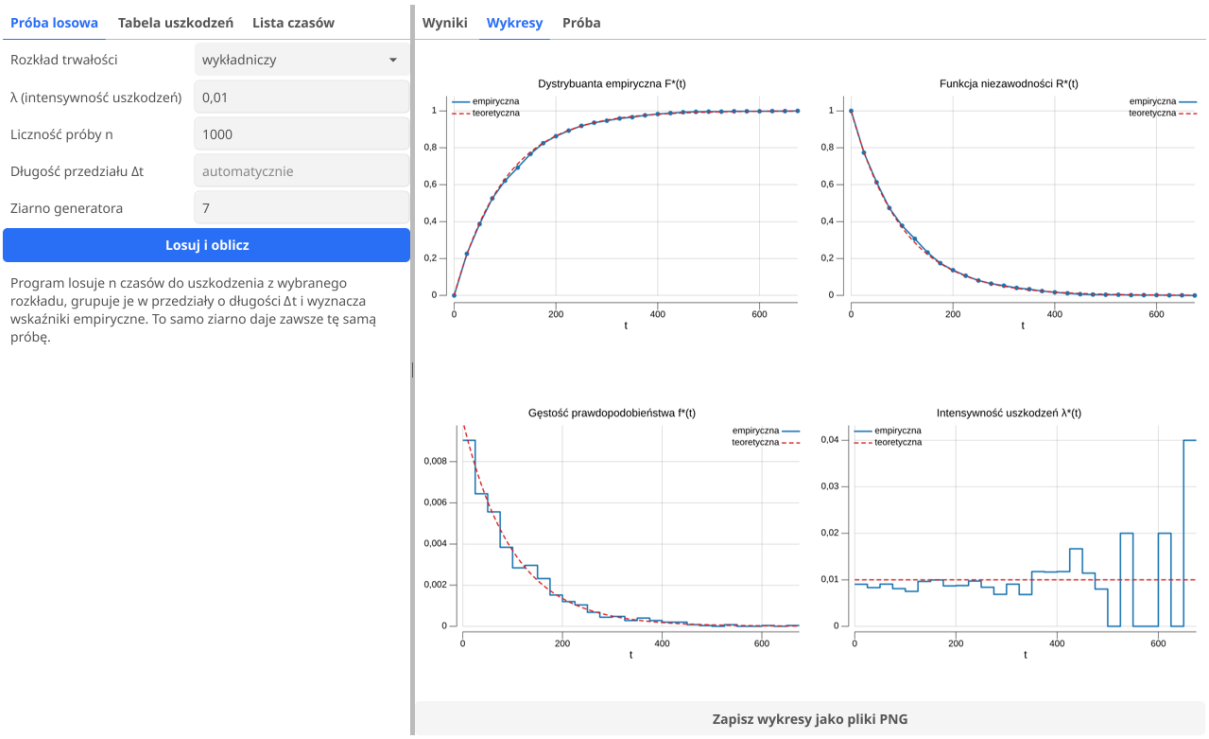
Znak „—” w kolumnie $\lambda^*(t_i)$ oznacza wartość nieokreśloną: na początku przedziału nie było już elementów zdatnych. Jeżeli nie wszystkie elementy uległy uszkodzeniu, E^*T ze wzoru (5-15) nie jest wyznaczana, a wartość z funkcji $R^*(t)$ jest oszacowaniem od dołu.



Rysunek 10: Zakładka „Wyniki” dla próby losowej

5.7 Wykresy

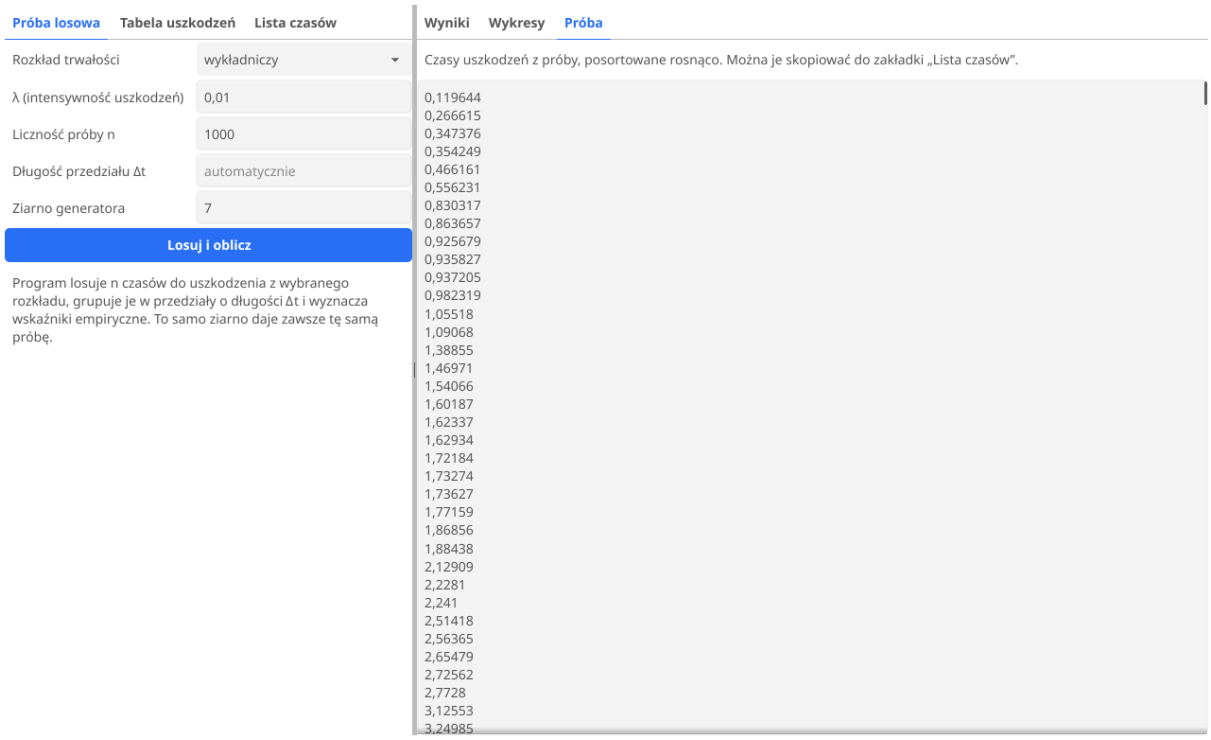
Zakładka „Wykresy” (rys. 11) pokazuje przebiegi $F^*(t)$, $R^*(t)$, $f^*(t)$ i $\lambda^*(t)$. Dystrybuanta i funkcja niezawodności są narysowane jako punkty w chwilach t_i połączone odcinkami. Gęstość i intensywność uszkodzeń dotyczą całych przedziałów, dlatego są narysowane jako wykresy schodkowe. Dla próby losowej czerwona linia przerywana pokazuje przebieg teoretyczny rozkładu, z którego losowano.



Rysunek 11: Zakładka „Wykresy” dla próby losowej

5.8 Próba

Zakładka „Próba” (rys. 12) zawiera pojedyncze czasy uszkodzeń posortowane rosnąco – wylosowane albo wpisane w zakładce „Lista czasów”. Można je zaznaczyć i skopiować, na przykład po to, by wkleić je do zakładki „Lista czasów” i przeliczyć z inną długością przedziału. Dla danych z tabeli uszkodzeń zakładka jest pusta, ponieważ pojedyncze czasy nie są wtedy znane.



Rysunek 12: Zakładka „Próba" z wylosowanymi czasami

5.9 Zapis wyników

- **„Zapisz wyniki do pliku CSV"** (zakładka „Wyniki") zapisuje tabelę wyników oraz wartości $n(0)$ i E^*T. Plik ma domyślną nazwę `results.csv`, średnik jako separator kolumn i przecinek dziesiętny, więc otwiera się bezpośrednio w polskiej wersji programu Excel.
- **„Zapisz wykresy jako pliki PNG"** (zakładka „Wykresy") zapisuje cztery wykresy do wskazanego folderu jako `cdf.png` (dystrybuanta), `reliability.png` (funkcja niezawodności), `density.png` (gęstość) i `hazard.png` (intensywność uszkodzeń).

Oba przyciski działają dopiero po wykonaniu obliczeń.

5.10 Komunikaty o błędach

Jeżeli dane są niepoprawne, program wyświetla okno z opisem błędu (rys. 8) i nie zmienia wyników. Komunikat wskazuje pole formularza, numer wiersza tabeli albo pozycję na liście czasów, w której wykryto błąd. Po zamknięciu okna przyciskiem „OK" należy poprawić dane i ponowić obliczenia. Przykłady komunikatów zawiera tab. 7.

5.11 Budowanie programu z kodu źródłowego

Do zbudowania programu potrzebne są: Go w wersji 1.26 lub nowszej oraz kompilator języka C, którego wymaga biblioteka Fyne. Plik wykonywalny buduje się na tym systemie, na którym ma działać, poleceniem wydanym w katalogu głównym programu (tab. 9). Wynikiem jest zawsze jeden plik wykonywalny.

Tabela 9: Budowanie programu w poszczególnych systemach

System	Dodatkowe wymagania	Polecenie
Windows	gcc (MinGW-w64)	`go build -ldflags "-H=windowsgui -s -w" -o empirical-reliability.exe .`
Linux	pakiety `gcc`, `libgl1-mesa-dev`, `xorg-dev`, `libxkbcommon-dev`	`go build -ldflags "-s -w" -o empirical-reliability .`
macOS	narzędzia wiersza poleceń Xcode	`go build -ldflags "-s -w" -o empirical-reliability .`

Literatura

[1] *Niezawodność urządzeń elektronicznych*. Praca zbiorowa pod red. M. Prażewskiej. Wydawnictwa Komunikacji i Łączności, Warszawa 1987, s. 208–213.

[2] *The Go Programming Language*. Dokumentacja języka i biblioteki standardowej, `https://go.dev/doc/`.

[3] *Fyne*. Biblioteka graficznego interfejsu użytkownika dla języka Go, `https://fyne.io`.

[4] *Gonum Plot*. Biblioteka do rysowania wykresów dla języka Go, `https://github.com/gonum/plot`.